

Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study

ANIL KOYUNCU, Computer Engineering, Bilkent University, Ankara, Turkey

Accurate classification of issues is essential for effective project management and timely responses, as the volume of issue reports continues to grow. Manual classification is labor-intensive and error-prone, necessitating automated solutions. While large language models (LLMs) show promise in automated issue labeling, most research focuses on broad categorization (e.g., bugs, feature requests), with limited attention to fine-grained categorization. Understanding specific bug types is crucial, as different bugs require tailored resolution strategies.

This study addresses this gap by evaluating LLMs and prompt engineering strategies for fine-grained bug report categorization. We analyze 221,184 fine-grained bug report category labels generated by selected LLMs using various prompt engineering strategies for 1,024 bug reports. We examine how LLMs and prompt engineering influence output characteristics, control over outputs, and categorization performance. Our findings highlight that LLMs and prompt engineering significantly impact output consistency and classification capability, with some yielding consistent results and others introducing variability. Based on these findings, we analyze the agreements and disagreements between LLM-generated labels and human annotations to assess category correctness. Our results suggest that examining label consistency and discrepancies can serve as a complementary method for validating bug report categories, identifying unclear reports, and detecting misclassifications in human annotations.

CCS Concepts: • **Software and its engineering** → **Software post-development issues; Maintaining software; Software evolution;**

Additional Key Words and Phrases: Prompt Engineering, Large Language Models, Automatic Bug Report Classification, Label correctness

ACM Reference format:

Anil Koyuncu. 2026. Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study. *ACM Trans. Softw. Eng. Methodol.* 35, 3, Article 67 (February 2026), 47 pages.

<https://doi.org/10.1145/3736408>

1 Introduction

Understanding the nature, root cause, and intrinsic characteristics of software bugs is crucial for effective bug management. While advances in automation have contributed to many aspects of the bug management process, development teams still rely heavily on user-reported bugs as a vital

This study was supported by Scientific and Technological Research Council of Turkey (TUBITAK) under the Grant Number 124E112.

Author's Contact Information: Anil Koyuncu (corresponding author), Computer Engineering, Bilkent University, Ankara, Turkey; e-mail: anil.koyuncu@cs.bilkent.edu.tr.



This work is licensed under Creative Commons Attribution-NonCommercial-ShareAlike International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1557-7392/2026/2-ART67

<https://doi.org/10.1145/3736408>

ACM Transactions on Software Engineering and Methodology, Vol. 35, No. 3, Article 67. Publication date: February 2026.

resource for managing and resolving software issues [50]. When a bug report is submitted, a series of bug life-cycle events are triggered before the development team starts working on the issue. A critical step in this process is the categorization of bug reports, which helps teams efficiently prioritize and route issues based on their type, severity, and impact. Effective categorization allows development teams to focus on the most critical bugs first, ensuring efficient allocation of resources and timely resolutions [13], while also helping to reduce reassignment of previously assigned bugs to others [44]. However, manual categorization can be labor-intensive and error-prone [37]. Bug reports can vary significantly in detail, language, and structure, ranging from well-documented issues with clear steps to reproduce, to vague or incomplete descriptions [15, 101]. This variability can lead to inconsistent categorization and misrouting of issues, further delaying the resolution process [37]. Moreover, the large volume of bug reports generated in major software projects makes it challenging for human triagers to consistently apply accurate labels. Human biases and fatigue can influence decision-making, leading to labeling errors that ultimately reduce the effectiveness of the bug management process [88].

To cope with the challenges of manual categorization by developers, researchers are increasingly exploring automated issue report classification systems. These systems utilize various techniques, including machine learning and natural language processing, to classify bug reports based on multiple criteria, such as, assessing severity or priority [1, 62], determining potential impact [87], distinguishing report types (e.g., bug, feature request) [6, 13, 46], identifying fault triggers [27, 89], assigning specific bug types or root causes [38, 74], and determining security relevance [31]. With the advent of transformers and **Large Language Models (LLMs)**, recent studies [19, 20, 22, 43, 51] have leveraged **Pre-Trained Language Models (PLMs)** for automatic issue labeling, and mostly focus on classifying issues into distinct types, including bugs, feature requests, and questions. The emergence of LLMs marks a significant evolution in software development by enabling more automated, intelligent, and efficient processes. However, despite the promising capabilities of advanced LLMs in general issue classification, we are concerned in several main dimensions:

- *Fine-Grained Categorization*: While existing LLM techniques offer various categorization options [19, 20, 22, 43], they often lack the fine-grained classification needed to distinguish between different types of bug reports. A broad categorization might simply label a report as a “bug,” but a more detailed approach would identify specific issues, such as UI problems, performance bottlenecks, security vulnerabilities, configuration errors, database issues, or network-related bugs. Understanding these distinctions is critical because different bug types necessitate tailored resolution strategies. Misclassifying a bug can delay the triaging process or lead to inefficiencies. For instance, a performance issue may require profiling and optimization, whereas a security vulnerability demands immediate patching and potentially public disclosure. Therefore, exploring the potential of LLMs for fine-grained bug report categorization is an area in need of further investigation.
- *LLM Diversity*: The potential of LLMs for automatic bug report categorization has primarily been studied using proprietary GPT models from OpenAI [20, 22, 51], while open-source alternatives remain largely under-explored. Exploring models open-source alternatives like Llama, Mistral, Gemini could provide a broader understanding and applicability of LLMs for bug report categorization.
- *Prompting Strategy*: Recent studies on bug report categorization primarily focus on zero-shot and few-shot learning [20, 22], as well as **Chain-of-Thought (CoT)** and **Tree-of-Thoughts (ToT)** [51] techniques. While these approaches have demonstrated effectiveness, other promising prompting strategies [2, 98]—demonstrated to improve performance in various

software engineering tasks [39, 70]—have yet to be explored in the context of bug report categorization.

- Dataset Label Correctness*: Wu et al. [88] caution that mislabeled instances in bug report datasets negatively impact prediction model performance. However, the capability of LLMs to identify and rectify mislabeled bug reports has not been thoroughly explored.

This Article: We seek to fill these gaps through a comprehensive, large-scale assessment of LLMs using different prompting strategies and explore how they can be applied for fine-grained bug categorization. Unlike previous studies [19, 20, 22, 43] that mainly classify broader issue types, such as feature requests and questions, or on classifying bug reports into general categories like **Bohrbug (BOH)**, **Aging-Related Bug (ARB)**, **Non-Aging-Related Mandelbug (NAM)**, and non-bug [51], our research focuses on how LLMs can categorize bug reports into detailed categories that accurately represent specific subtypes of bugs. We specifically explore how LLM diversity and prompting strategies influence the capabilities of LLMs in automated bug report classification, while also assessing dataset *label correctness* by leveraging LLM-generated labels to identify potential misclassifications or unclear bug reports.

We evaluate the automatic fine-grained bug report categorization task on six widely used open-source LLMs—Mistral from MistralAI, Gemma2 from Google, Llama3 from Meta, Aya from Cohere, Qwen2 from Alibaba, and Starling from Berkeley—using prompt engineering strategies, which focus on *nine distinct prompt types* and four output format instructions. Our comparative analysis evaluates the categories generated by these LLMs against human-annotated categories, using the taxonomy and dataset provided by Catolino et al. [13].

Our main contributions are as follows:

- (1) We provide insights on the capabilities of LLMs for fine-grained bug report categorization based on a comprehensive analysis of the impact of prompt engineering. To the best of our knowledge, this is the first empirical study in the literature that evaluates open-source LLMs for fine-grained bug report categorization.
- (2) We assess widely used prompt engineering techniques—such as *CoT*, and *ToT* prompting—as well as explore recent techniques, including **Automatic Prompt Engineer (APE)**, **Plan-and-Solve (PS)**, **Least-to-Most (LTM)**, **Self-Consistency (SELF)**, and **Multi-Agent Debate (MAD)** prompting, for fine-grained bug report categorization. Additionally, we introduce a new prompt, **Iterative Inspector Consensus (ITER)**, designed to simulate the multi-step bug report categorization process used by human evaluators, as outlined in Catolino et al. [13].
- (3) We analyze the consistency of LLM output formats and demonstrate that LLMs often deviate from expected output format in the context of bug report categorization. To address this, we present a method based on prompt engineering and lightweight post-processing aimed at increasing the likelihood of retrieving accurate bug categories from model outputs.
- (4) We evaluate the agreements and disagreements among bug category labels generated by LLMs across various settings to assess dataset *label correctness*. Our analysis examines instances where model-generated labels align with or diverge from each other, helping to identify potential misclassifications (where models agree on an incorrect categorization) and inconsistencies (where different models provide conflicting categorizations). Our findings suggest that by analyzing these discrepancies, model-generated labels can serve as a complementary validation method, identifying ambiguous or poorly detailed reports, flagging potential errors in human-assigned labels, and assessing the correctness of dataset labels.
- (5) To support ongoing and future research in this field, we are releasing a comprehensive benchmark that includes (1) a labeled dataset comprising 221,184 LLM-generated bug report

categories, produced by six LLMs using nine prompt types and four output configurations for 1,024 bug reports, (2) the dataset of corrected bug report categories based on our validation method, which improves upon human annotations, and (3) evaluation scripts. These artifacts are available at: <https://zenodo.org/records/14195747>.

The remainder of this article is structured as follows: Section 2 details the background of LLMs for automatic bug report categorization and prompt engineering. Section 3 outlines our experimental setup, followed by Section 4, which discusses the evaluation results. While Section 5 addresses potential study threats and limitations, Section 6 discusses the findings and their implications, and Section 7 reviews related work. Section 8 offers concluding remarks.

2 Background

In this section, we introduce the fundamental concepts essential for understanding our article. We briefly cover LLMs for automatic bug report categorization along with prompt engineering, which constitutes the central ideas of this study.

2.1 LLM-Based Bug Categorization

LLMs are advanced neural networks primarily based on the Transformer architecture [79], trained on large-scale text corpora and code using self-supervised techniques like masked language modeling and causal language modeling [34, 58, 68]. These models come in various architectures, including encoder-only like BERT [25], decoder-only like GPT [12, 93], and encoder-decoder models like T5 [64]. They are often enhanced with methods like reinforcement learning from human feedback [10, 100] to better align with human intentions. When interacting with these models, the instructions provided along with any input data constitute what is referred to as a prompt [5, 81]. LLMs have been notably applied across a wide array of software engineering activities, including coding, design, requirements analysis, repair, refactoring, performance improvement, documentation, and analytics [39, 70].

Despite the growing body of literature on the usage of LLMs in automated issue classification [3, 4, 7, 11, 19, 20–22, 26, 36, 57, 65, 72, 78], most existing studies predominantly utilize transformer models in a supervised learning context, focusing on categorizing issues into distinct types, such as bugs, feature requests, and questions. As a result, the effectiveness of LLMs in fine-grained bug categorization remains under-explored. This study aims to address this gap by exploring the application of open-source LLMs and various prompt engineering techniques for fine-grained bug categorization, thereby providing a comprehensive analysis of their effectiveness.

2.2 Prompt Engineering

Prompt engineering is a technique aimed at improving model performance through the use of task-specific instructions called prompts. This strategy allows LLMs to effectively engage in downstream tasks solely based on the provided prompts, directing model behavior without the necessity of modifying model parameters [5, 58, 66, 81]. This developing area focuses on designing prompts that guide LLMs to generate specific outputs, ranging from broad descriptions to detailed specifications. The relevance of prompt engineering lies in its ability to significantly impact model performance: specific prompts lead to more accurate responses, while general prompts may produce less relevant outputs [71]. Furthermore, refining prompt instructions can better control the output format, reducing the need for post-processing and directly enhancing performance evaluation [54]. Various studies [29, 59, 60, 82] have leveraged LLMs and prompt engineering for automated software engineering tasks, commonly using basic approaches such as zero-shot [48] and few-shot learning [12]. Recently, research has broadened prompt engineering to include techniques such as

CoT [86], SELF [85], and ToT [85]. To build on these advancements, our study explores a range of prompt engineering strategies to evaluate and enhance the fine-grained categorization of bug reports using LLMs, as outlined below:

Standard Prompting (SIMP): The practice of providing LLMs with tasks or questions without any examples or additional guidance. This approach evaluates the model's baseline ability to handle the task with minimal instruction. In our study, we prompt LLMs to categorize bugs without providing examples or guidance, aiming to assess their raw abilities for this task.

CoT Prompting: CoT prompting [86] is designed to improve reasoning by prompting the model to break down its thought process into logical steps. We follow the prompt "Let's think step by step" from Kojima et al. [48] which serves as a general template for guiding LLMs in reasoning tasks.

APE: APE [96] generates optimized prompts to enhance model performance on zero-shot and few-shot learning tasks by automatically finding effective CoT prompts. Following the insight from Zhou et al. [96], which demonstrated that the APE-engineered prompt "Let's work this out in a step-by-step way to be sure we have the right answer" can improve LLM zero-shot CoT performance, we incorporate this prompt in our evaluation to enhance model reasoning and accuracy.

LTM Prompting: We incorporate LTM prompting [95] to help LLMs tackle tasks that exceed the complexity of those in the prompt examples. This strategy follows a two-step process: first, it breaks down the problem into manageable subcomponents (top-down), and then builds solutions from the ground up (bottom-up) to resolve each part progressively. According to findings from the original study, LTM prompting outperformed standard and CoT prompting in tasks that require advanced reasoning, including symbolic manipulation, compositional generalization, and mathematical problem-solving.

PS Prompting: Introduced by Wang et al. [84], PS prompting is a new zero-shot CoT method designed to enhance the problem-solving capabilities of LLMs. The method encourages the model to first devise a structured plan for solving a given task by breaking it down into smaller sub-tasks, and then carrying out the sub-tasks according to the plan.

ToT Prompting: Yao et al. [92] and Long [53] proposed a new technique that generalizes over CoT prompting and encourages exploration over thoughts that serve as intermediate steps for general problem-solving through a deliberate reasoning process. In this study, we adopt a single-prompt approach inspired by Hulbert [42], which applies the ToT concept by prompting the model to evaluate and integrate intermediate steps within a single prompt, aiming to enhance its general problem-solving abilities through multi-step reasoning.

MAD Prompting: The MAD framework proposed by Liang et al. [52] prompts multiple language model agents to engage in a back-and-forth process, simulating a form of negotiation or debate. In this process, each agent has the opportunity to respond to, challenge, or build upon the previous agent's argument. This iterative exchange allows for the exploration of different perspectives before a final solution is derived, with a designated judge moderating the debate to assess the quality and relevance of the arguments. In this study, we adopt a single prompt proposed by Sun et al. [73] to simulate MAD within a single prompt.

SELF Prompting: SELF prompting [85] aims to reduce randomness in the outputs of LLMs by generating multiple reasoning results and selecting the most consistent answer among them. Inspired by the 1-shot SELF prompt template,¹ our study adapts a single prompt that generates multiple solutions, each with distinct reasoning paths, and aggregates these outputs to achieve a final consensus.

ITER Prompting: Inspired by previous prompting strategies, we design this new prompt type to simulate the iterative bug report categorization process outlined by Catolino et al. [13].

¹<https://www.prompthub.us/blog/self-consistency-and-universal-self-consistency-prompting>.

Section 3.3 provides detailed descriptions of the designed prompts, including the prompt templates, which follow best practices in prompt engineering and insights from prior research. Additional examples of prompt engineering techniques can be found in the replication package.

Numerous studies have leveraged LLMs and prompt engineering to address ASE tasks [39, 70]. In the context of bug report categorization, previous work has explored zero-shot and few-shot learning approaches [20, 22], as well as CoT and ToT techniques [51]. Our study builds on this work, expanding on these approaches by exploring additional prompt engineering strategies to enhance the fine-grained categorization of bug reports using LLMs.

3 Study Setup

This section outlines the **Research Questions (RQs)**, the evaluation benchmark and bug report categories, the design of the prompts employed in the experiments, and the selection of LLMs for our study.

3.1 RQs

To frame our research, we propose to answer the following RQs:

RQ1: What are the characteristics of the generated outputs across LLMs and prompts? We focus on analyzing the characteristics of the generated output of the LLMs across different models and prompt types. Concretely, we analyze the LLMs' ❶ adherence to output instructions, ❷ their ability to generate bug categories consistent with the initially provided set of categories, and ❸ we examine generated outputs to identify potential common patterns among them. By evaluating these aspects in this RQ, we aim to reveal each model's strengths and weaknesses in adhering to instructions, maintaining consistent categorization, and producing predictable output styles, thereby guiding improvements in prompt engineering and model selection.

RQ2: To what extent can the output format of LLMs be controlled using prompt engineering and lightweight post-processing? We explore the extent to which the output format of LLMs can be controlled so that we can extract bug report categories automatically in a unified way. To achieve this, ❶ we first focus on prompt engineering, specifically examining how output block configurations can enhance the model's ability to follow instructions accurately. ❷ We implement a post-processing technique using extended regular expression parsing to capture variations in the model's output. By combining these strategies, we aim to improve the alignment of generated outputs with expected categories, ultimately facilitating a more efficient and reliable categorization process.

RQ3: How do prompt engineering strategies impact the performance of LLMs in categorizing bug reports? We empirically quantify the impact of prompt engineering strategies on the performance of LLMs in categorizing bug reports. Our analysis is organized into three main sections: ❶ We evaluate how different prompt engineering strategies influence overall model performance, measured by **precision, recall, and F1-scores**. ❷ We assess the performance metrics—including **precision, recall, and F1-scores**—specific to each issue class, providing insights into the models' ability to categorize various types of bugs. ❸ We examine how these strategies impact the alignment of model-generated outputs with **human annotations**, measuring agreement using **Cohen's Kappa**.

RQ4: To what extent can model-generated bug categorizations serve as a validation approach for human-labeled bug categories? We investigate the potential of using model-generated bug categorizations as a systematic approach to validate the correctness and reliability of human-applied labels. Specifically, ❶ we assess how well these automated labels align with human ones by measuring inter-model agreement to evaluate the consistency of different models across various configurations in our study. ❷ We analyze their ability to detect errors or inconsistencies by examining the agreements and disagreements among bug category labels generated by different models to identify inconsistencies and consistencies. This analysis serves to validate bug report categorizations, where

Table 1. List of Bug Categories and Their Descriptions

Issue Type	Description
Configuration issue	Bugs related to build configuration files, such as issues with external libraries that need updating or fixing and incorrect directory or file paths in XML or manifest artifacts.
Network issue	Bugs caused by connection or server problems, which may result from network issues, unexpected server shutdowns, or improper use of communication protocols within the source code.
Database-related issue	Problems involving the connection between the main application and a database.
GUI-related issue	Stylistic errors, such as screen layouts, element colors and padding, text box appearance, and buttons, as well as unexpected failures appearing to the users as unusual error messages.
Performance issue	Issues related to memory overuse, energy leaks, and methods causing endless loops.
Permission/deprecation issue	Issues involving the presence, modification, or removal of deprecated method calls or APIs; unused API permissions; and unexpected behavior when methods of external APIs are called.
Security issue	Vulnerability and other security-related problems, including the need to reload certain parameters and removal of unused permissions that might decrease the overall reliability of the system.
Functional issue	Bugs in core application logic causing unexpected behavior, exceptions, problems with return values, and unexpected crashes due to issues in the logic (rather than, e.g., the GUI) of the program.
Test code-related issue	Bugs appearing in test code, including problems due to running, fixing, or updating test cases, intermittent tests, and the inability of a test to find de-localized bugs.

inconsistencies may indicate misclassifications or a lack of understanding of the bugs, while consistencies suggest reliable patterns that enhance categorization accuracy, and ③ We explore their potential to improve the accuracy and reliability of human-labeled bug categories by conducting a manual review of the selected model-generated labels that show identified inconsistencies and consistencies, comparing them to human-assigned categories. This review involves analyzing a sample of bug reports to assess the alignment between model-generated labels and those assigned by human annotators. By addressing discrepancies and agreements, our goal is to establish a robust validation approach for human-labeled bug categories.

3.2 Evaluation Benchmark

In this study, we use the dataset and the taxonomy developed by Catolino et al. [13]. They created a taxonomy of bug types through an iterative analysis of 1,139 randomly sampled bug reports from 119 software projects across three large ecosystems: Mozilla, Apache, and Eclipse. The taxonomy, which includes nine bug categories, was refined through expert reviews, discussions, and re-categorization. To validate its robustness, five experienced industrial developers assessed the taxonomy. The dataset covers a wide range of issues, including GUI problems and configuration errors. Following the taxonomy of bug types from Catolino et al., we summarize the bug categories and their corresponding descriptions, as outlined in their work, in Table 1.

After reviewing the Catolino dataset [13], we identify areas that require further curation for our specific analysis. Catolino et al. state that their taxonomy is based on 1,139 bug reports after excluding 141 from an initial set of 1,280. Starting with these 1,139 reported bug reports, we apply our own data curation process to ensure consistency and relevance, which includes the following steps:

- (1) *Retrieve Full Data*: The original dataset primarily contains summaries and links. We first crawled the corresponding bug tracking systems (like Bugzilla) using the provided links to retrieve the full description text for each bug report, providing essential context. We also retrieve and standardize the original “classification” field provided by Catolino et al. (lowercasing and stripping whitespace).
- (2) *Handle Missing Descriptions*: We removed bug reports where the full description could not be retrieved or was missing (NA). This step filtered out 21 reports, leaving 1,118.

Table 2. Dataset: Distribution of Bug Reports by Category and Ecosystems

Categories	Apache	Eclipse	Mozilla	Total
Configuration issue	50	52	66	168
Database-related issue	4	18	5	27
Functional issue	161	163	111	435
GUI-related issue	69	52	61	182
Network issue	18	11	13	42
Performance issue	15	13	11	39
Permission/deprecation issue	10	23	4	37
Security issue	18	2	7	27
Test code-related issue	16	39	12	67
Total	361	373	290	1,024

- (3) *Filter by Description Length*: To exclude potentially uninformative reports, we removed entries with very short descriptions (fewer than 50 characters). This removed 17 reports, resulting in 1,101. We also removed 10 reports with excessively long descriptions to maintain focus, leaving 1,091 reports.
- (4) *Remove Duplicates*: We then identified and removed duplicate bug reports based on their content (e.g., identical summaries and descriptions). This step removed 20 duplicate entries, leaving 1,071 unique reports.
- (5) *Filter Non-Bug Entries*: We observed reports related to code reviews rather than bugs, often starting with “review:”. We filtered out entries matching this pattern (e.g., https://bugzilla.mozilla.org/show_bug.cgi?id=159576). This removed 44 such reports, resulting in 1,027 relevant bug reports.
- (6) *Exclude Test Set*: Finally, we excluded three specific reports that were previously separated for internal testing purposes during prompt development, leading to our final dataset size of **1,024 bug reports**.

These curation steps represent our best-effort approach to ensure the dataset used for our experiments is clean, consistent, and focused on relevant bug reports with sufficient context. Table 2 shows the statistics of our final dataset after curation, including the number of bug reports, their distribution across bug categories, and ecosystems.

3.3 Prompt Design

Prompt Template: To evaluate the impact of prompt engineering on LLMs in the task of bug categorization, we structure our experimental setup to assess how different prompt designs influence model performance. By incorporating various reasoning strategies—such as CoT, ToT, and PS—and defining clear output formats, we aim to understand which aspects of prompt engineering enhance the model’s ability to generate accurate and well-organized responses. Central to this approach is the use of a consistent base prompt template.

We use a structured prompt template, as shown in Figure 1. This base template structure was designed drawing upon common practices in prompt engineering literature, ensuring key components like task context, explicit instructions, category definitions, and desired output format are clearly delineated. The prompt is carefully crafted to provide clear instructions, a well-defined input, and an expected output format, thereby ensuring consistency across the different experiments and serving as a controlled baseline. While we acknowledge that guaranteeing the absolute optimality of any single template is challenging, our goal was to establish a reasonable and representative foundation

Categories	# Categories PLACE HOLDER
Instructions	# Instructions 1. You will be given a bug report with a title and description, and your task is to assign the bug report into categories one of the following categories: [database-related issue, test code-related issue, security issue, configuration issue, network issue, gui-related issue, functional issue, permission/deprecation issue, performance issue] 2. PLACE HOLDER 3. Respond in the following Markdown structure:
Output Format	```\n---\n# Category\nHere, indicate only the name of decided category.\n---\n```
Input Format	# Bug Report ## Title PLACE HOLDER ## Description PLACE HOLDER

Fig. 1. Prompt template.

upon which the effects of different prompting strategies (RQ1) and output configurations (RQ2) could be reliably compared. The components of this prompt template include:

- *Categories*: This section defines the specific categories for bug classification, along with their corresponding descriptions, ensuring the model has a predefined set of outcomes to choose from. The PLACE HOLDER is used for brevity, and during execution, it is replaced with the actual category labels and descriptions listed in Table 1.
- *Instructions*: The instructions provide guidance on how the model should categorize the bug report, outlining the task and any relevant context needed for accurate classification. The PLACE HOLDER is employed to inject different prompting strategies (as listed earlier in our study) into the template. In Step 2 of the instructions, the content will be dynamically replaced according to the chosen prompt type during execution.

We examine nine distinct prompting strategies to assess and enhance the performance of LLMs on bug categorization tasks, each tailored to explore different aspects of the model’s understanding and reasoning capabilities. From the prompting strategies listed above, we identify prompt phrases and directives that reflect prompting strategies aimed at encouraging systematic reasoning. We carefully design our prompts using explicit instructions based on the existing literature and integrate into our prompt templates to guide the model’s reasoning process. Table 3 displays the types of prompts used, along with the specific instructions incorporated into each template.

- *Output Format*: The output format specifies the expected structure of the model’s response. Initially, we ask models to output only the category for the given bug report. To enhance consistency and reduce ambiguity in the categorization, we also experiment with additional output formats, where models are asked to provide their reasoning process and explanations. To ensure alignment with the prompt type specified instructions, variations in the process output instructions are introduced depending on the selected prompt type. Table 4 illustrates

Table 3. Prompt Types and Corresponding Explicit Instructions Integrated into the Templates

Prompt Type	Instruction
COT	Let's think step by step.
APE	Let's work this out in a step-by-step way to be sure we have the right answer.
LTM	First, decompose the task into several sub-tasks that need to be solved, and then solve each task step by step.
PS	First, understand the task and devise a plan to complete the task. Then, carry out the plan and complete the task step by step.
TOT	Follow these steps: All experts will write down 1 step of their thinking and then share it with the group. Then all experts will go on to the next step, and so on. If any expert realizes they're wrong at any point, then they leave. After this, the group will discuss and come to a consensus on the most appropriate category.
MAD	Follow these steps: three experts are discussing the bug report, trying to categorize it step by step, and make sure the result is correct.
SELF	Given the task, generate multiple solutions using diverse reasoning paths and aggregate the final solutions to reach a final conclusion.
ITER	Follow these steps: Iteration 1: Initial Categorization by Multiple Inspectors Each inspector (you will simulate their perspectives) individually analyzes the title and description of the bug report. Each inspector assigns an initial category based on their analysis. Iteration 2: Discussion and Consensus Inspectors discuss the initial categorizations. If discrepancies arise, they debate the appropriate category, considering potential overlaps or ambiguities. Inspectors reach a consensus on the most accurate category. Iteration 3: Final Categorization and Validation They validate that the final categorization accurately reflects the issue described in the bug report.
SIMP	None

Table 4. Output Instructions Provided to Models for Each Prompt Type in the Study

Prompt Type	Instruction
Process Block	
LTM, COT, PS, APE	# Process Here, describe the step-by-step thinking process in a few sentences.
SELF	# Process Here, describe each reasoning path and aggregated reasoning process in a few sentences.
ITER, MAD, TOT	# Process Here, describe the inspection process iterations in a few sentences, summarizing each expert's thoughts and how the group reached a final consensus.
SIMP	# Process Here, describe the process in a few sentences.
Explanation Block	
ALL	# Explanation Here, provide the reasoning for why the chosen category is the most appropriate.
Category Block	
ALL	# Category Here, indicate only the name of decided category.

the mapping between prompt types and the corresponding output instructions for all the output formats considered in our study.

- *Input*: This section provides the actual bug report to be classified, serving as the input data for the model. The PLACE_HOLDER values are replaced with the bug report's title and description during execution.

3.4 Selected LLMs

The field of LLMs is evolving rapidly, offering a range of capabilities and performance levels for various automated software engineering tasks [29, 39]. Recognizing the importance of evaluating

Table 5. Overview of LLMs Used in the Study

Model Name	Architecture	Size (Billions)	Context Length	Quantization	Release Date	Ollama Commit
gemma2:9b-instruct-q4_0	Gemma2	9B	8,192	Q4	21 February 2024	ff02c3702f32
qwen2:7b-instruct-q4_0	Qwen2	7.6B	32,768	Q4	3 August 2023	e0d4e1163c58
mistral:7b-instruct-v0.3-q4_0	Llama	7B	32,768	Q4	22 May 2024	b17615239298
llama3.1:8b-instruct-q4_0	Llama	8.0B	8,192	Q4	23 July 2024	42182419e950
aya:8b-23-q4_0	Command-R	8.0B	8,192	Q4	23 May 2024	7ef8c4942023
starling-lm:7b-beta-q4_0	Llama	7B	8,192	Q4	20 March 2024	39153f619be6

these models, we selected six open-source LLMs that have not been previously assessed for automated bug report categorization. The selection of these models is based on several dimensions, as outlined below:

- (1) *Model Diversity*: We selected models from diverse architectures, including *Gemma*, *Qwen*, *Llama*, and *Command-R*, to ensure a broad evaluation across different training methodologies. This diversity helps mitigate potential biases that might arise from relying on a single model or architecture.
- (2) *Model Size and Scalability*: The models selected range in size from 7B to 9B parameters, balancing computational efficiency and expressive power. This size range is suitable for academic and research settings, where resources are often limited, allowing meaningful experimentation without placing excessive demands on the computational infrastructure.
- (3) *Computational Efficiency*: All models support *Q4 quantization*, reducing memory and computational costs while maintaining performance. This enables the models to run efficiently on consumer-grade hardware, making the experiments accessible to researchers and practitioners without requiring large-scale computational resources.
- (4) *Recency and State-of-the-Art Performance*: We focus on models released in 2023 and 2024, reflecting the latest advancements in LLM technology at the time of submission. These models incorporate up-to-date optimizations, training datasets, and architectural improvements, ensuring their relevance to current research and real-world applications. By selecting these recent models, we ensure that our study evaluates the most current innovations in the field.
- (5) *Availability and Reproducibility*: All selected models are available on Ollama, an open-source platform to run and share LLMs locally. This ensures the reproducibility of our results and aligns with real-world deployment scenarios, where LLMs are often used in local environments.

Table 5 summarizes the details of the selected models.

- *Starling-LM*: Starling-7B [98] is a 7-billion-parameter language model developed using reinforcement learning from AI feedback with the goal of improving chatbot helpfulness. It leverages a GPT-4-guided dataset that enhances response quality by aligning outputs with human-like ranking preferences.
- *Llama*: Meta’s Llama models [77] are open-source, pre-trained on 1.4 trillion tokens, with sizes ranging from 7B to 405B parameters. The latest version, Llama3.1, includes models in 8B, 70B, and 405B sizes, featuring a longer context length of up to 128K tokens. It excels in general knowledge, math, tool use, and multilingual translation, making it suitable for applications like text summarization, conversational agents, and coding assistance.
- *Mistral*: Mistral-7B [45] is a highly efficient 7-billion-parameter model known for outperforming some larger models due to its optimized architecture. It achieves top benchmarks in mathematics, code generation, and reasoning, focusing on computational efficiency and task-specific precision.

- *Gemma*: Gemma is a family of lightweight, open LLMs from Google. Gemma 2 [75], the latest addition to this family, includes models ranging from 2 billion to 27 billion parameters. These models excel in reasoning and text generation, with particularly strong results on benchmarks such as HumanEval, GSM8K, and AGIEval for math and code-related tasks.
- *Aya*: Aya, released by Cohere, is a multilingual model that follows instructions in 101 languages. Built with a diverse dataset, handle a wide range of linguistic contexts, and shows strong performance across multilingual benchmarks. Aya 23 [9] builds on the recent release of the Aya model [102], is a new family of state-of-the-art, multilingual, generative large language research model (LLM) covering 23 different languages. The model shows strong performance across multilingual benchmarks in complex tasks, such as natural language understanding, summarization, and translation, across a wide linguistic spectrum.
- *Qwen*: Qwen is a series of open-source models developed by Alibaba, includes both general-purpose and chat-focused models with parameter sizes ranging from 0.5 to 72 billion. The version Qwen2 [91] demonstrates strong capabilities in natural language understanding, question answering, coding, mathematics, scientific knowledge, reasoning, and multilingual tasks.

All models used in this study were obtained from checkpoints available on Ollama, an open-source application that enables users to run, create, and share LLMs locally on their systems. For our experiments, we used Ollama v0.3.6.² When available, instruction fine-tuned versions of the models were selected to enable the models to follow instructions and perform task-specific prompts more effectively. Additionally, the quantization level was set to Q4 to reduce computational load and allow the models to run on smaller hardware setups. To ensure a fair evaluation of all models, we standardized the context length to 8,192 tokens across all models, preventing any advantage from differing context window sizes. Additionally, to minimize the randomness of the results, the temperature is set to 0 during generation.

4 Experimental Results

4.1 RQ1: What Are the Characteristics of the Generated Outputs across LLMs and Prompts?

4.1.1 Adherence to Output Instructions.

Approach: To assess how well the LLMs follow the instructions in the prompt, we evaluate how well each model and prompt type combination adheres to the output instructions specified in the prompt template. As described in Section 3.3, the prompt template (Figure 1) specifies in the output format section, the expected structure for the models' responses. All prompts are instructed to respond in Markdown format, featuring a Markdown header category and specifying only the name of the chosen category as the content underneath the heading. We chose Markdown format because of its widespread usage, which increases the likelihood that LLMs are familiar with it, and ease of extracting information for automatic processing. We use regular expressions (regex) to extract category heading blocks from the LLMs' responses, which are expected to contain the bug report category as specified in the prompt instructions. Equation (1) illustrates the regex used for this extraction. To assess whether the LLM follows the output instructions, we evaluate its responses to determine if it generates a category block in Markdown format correctly:

$$(?:Category)(?:\n)*(.*?)(?=$|\n). \quad (1)$$

Results: Table 6 presents the results for cases where the "Category" section, which is expected to be formatted in Markdown with the heading # Category and the label in its content, is missing from

²<https://github.com/ollama/ollama/tree/v0.3.6>.

Table 6. Adherence to Output Instructions

Prompt Type	Model					
	Aya	Gemma2	Llama3.1	Mistralv0.3	Qwen2	Starling-LM
SIMP	49	982	1,018	0	0	13
COT	202	947	1,022	6	0	28
APE	137	908	1,024	0	0	45
LTM	3	812	1,024	0	0	44
PS	33	986	1,024	0	0	44
TOT	1	701	640	0	0	199
MAD	41	739	954	0	0	167
SELF	4	781	1,020	0	0	77
ITER	31	924	1,023	0	0	66
Total	501	7,780	8,749	6	0	683

The table shows the instances where the expected category section is absent from the LLM-generated output.

the LLM-generated output. We observe that adherence to output format instruction vary per model and prompt type. Qwen2 is the most successful model, strictly following the output instructions and generating a category section for all generated outputs, regardless of the prompt type used. Mistral follows closely behind, failing to generate the category section in only six responses using the COT prompt type. The Aya and Starling-LM models generally succeed in adhering to the output instructions, with only about 5% (501 out of 9,216) of cases failing for Aya and about 7% (683 out of 9,216) for Starling-LM. In contrast, the Gemma2 model fails to follow the output instructions in approximately 84% of cases (7,780 out of 9,216), while the Llama model fails in about 95% of cases (8,749 out of 9,216). In terms of prompt types, the ToT prompt type adheres better to output instructions than others for Aya, Llama, and Gemma2. Specifically, Aya had only one failure, while Llama and Gemma2 had 640 and 701 failures, respectively, out of 1,024 generations, the fewest among the prompt types for these models. Similarly, the SIMP prompt, which provides the least guidance, adheres best to the output instructions compared to other prompt types for Starling-LM.

Finding 1: There is significant variation in adherence to output instructions across different models, emphasizing the need for post-processing to reliably extract the bug category. While some models, like Mistral and Qwen2, consistently follow the output instructions, others, such as Llama and Gemma, show substantial inconsistency. Additionally, some prompt types such as TOT and SIMP are more effective at adhering to output instructions for different models. This inconsistency underscores the necessity of exploring different prompting strategies or post-processing steps to accurately identify and extract the bug report categories from the outputs of these models.

4.1.2 Adherence to Expected Categories.

Approach: We assess the adherence of the LLMs to the expected bug categories by analyzing whether the predicted labels align with the predefined bug categories from Catolino’s taxonomy. We discard responses that do not align with any category or list multiple categories based on exact-match criteria. The analysis focuses on data where models adhere to the identified output formats, ensuring that the evaluation is based on cases with confirmed format adherence. This allows for an accurate assessment of how well predicted categories match the desired ones.

Results: Table 7 displays the number of discarded responses—where the LLM predicted bug report category is not included in the predefined set of bug categories—for each prompt type across all the models. The analysis of discarded responses reveals notable differences in how models align with expected categories, especially when considering the proportions relative to the total

Table 7. Discarded Responses

Prompt Type	Model					
	Aya	Gemma2	Llama3.1	Mistralv0.3	Qwen2	Starling-LM
SIMP	114	0	2	0	7	450
COT	80	0	1	0	1	580
APE	102	1	0	0	9	603
LTM	266	6	0	1	11	545
PS	143	0	0	1	9	521
TOT	22	7	104	352	38	144
MAD	10	20	28	11	1	216
SELF	496	3	1	56	29	702
ITER	13	0	0	0	1	340
Total	1,246	37	136	421	106	4,101
Support	8,715	1,436	467	9,210	9,216	8,533

The table represents the total number of discarded responses due to misalignment with expected categories, excluding those already counted as non-adherent to the output format. The “Total” row indicates the number of discarded responses per model due to misalignment with expected categories. The “Support” row shows the total number of responses where the models adhered to the output format, which is the subset of data used for this analysis as calculated in Table 6.

support. While Aya and Starling-LM show high absolute numbers of discarded responses—1,246 and 4,101, respectively, Aya’s discarded responses amount to approximately 14% of its support (8,715), whereas Starling-LM’s discarded responses represent about 48% of its support (4,101/8,533). This indicates that while these models have a higher rate of adherence to the output format instructions—responding explicitly with #Category section—the category labels generated by these models do not align with the generation of expected categories at the same rate. Additionally, Aya and Starling-LM exhibit high numbers of discarded responses across various prompt types, particularly in SELF prompt the highest, indicating that certain prompt types may present particular challenges for these models in terms of alignment with expected categories. Mistralv0.3 and Qwen2 are the two models that followed the output formatting instructions at the highest rate in the previous experiment. When it comes to generating consistent bug report categories, they had approximately 5% and 1% discarded responses, respectively. Mistralv0.3 and Qwen2 display a relatively high number of discarded responses in specific prompt types, such as ToT and SELF. These prompt types, along with MAD and ITER adopt multi-step reasoning and generate multiple answers using diverse reasoning paths to decide the category of the bug reports. The high number of discarded responses in these prompt types highlights the difficulties in adhering to the expected categories in these cases. Gemma2 and Llama3.1 are the worst-performing models in terms of following output formatting instructions from the previous experiment. However, when evaluating label consistency, Gemma2, with a support of 1,436, has a remarkably low number of discarded responses (37), representing just about 2% of its total responses. In contrast, Llama3.1, despite having a moderate total count of discarded responses (136), shows a higher proportion of about 29% relative to its support (467). Gemma2 and Llama3.1 struggle primarily with TOT and MAD prompts, both of which simulate multiple experts to decide the category for the bug report.

Finding 2: Adherence to formatting instructions does not guarantee better category prediction, suggesting that factors like prompt type also influence model performance. Our observations highlight variability in discarded responses due to misalignment with expected categories across different models and prompt types. The data underscores the influence of prompt type and model-specific characteristics on aligning with expected categories, emphasizing the need for tailored approaches to improve consistency in achieving these alignments.

4.1.3 Common Output Formats.

Approach: We examine the generated outputs of the LLMs to identify common patterns among them and gain insights into how various models respond to different prompt types. We analyze the structure and consistency of the output formats by creating a representative sample of LLM responses through stratified sampling across 54 strata, which represent the combinations of six models and nine prompt types. Each prompt type-model combination generates 1,024 inferences, resulting in a total of 55,296 inferences. From this set, we focus on those that do not adhere to the output formatting instructions (Table 6) and those that fail to align with the expected categories (Table 7). We then manually analyze each of these strata to identify common output patterns of the LLMs. This approach enables us to understand how different LLMs respond to various prompt types and offers valuable insights into LLM output formats within the context of bug report categorization.

Results: The examination of these 54 strata reveals that, despite direct instructions in the prompts, the models generate outputs in varying formats for bug report categorization. The following summary highlights the common patterns we observe in the generated outputs across the different models:

- (1) *Direct Output:* This is the expected format as specified by the prompt: a Markdown header for the category, with only the name of the chosen category listed beneath the heading.
- (2) *Wrapped Category:* The bug category is surrounded by or wrapped in various forms of quotation marks, where each instance can use different quotation styles or symbols to indicate emphasis or formatting of the category name.
- (3) *Multiple Categories:* The model's response can contain multiple categories for the bug report in question.
- (4) *Out-of-Dictionary Category Labels:* The LLM's response suggests a bug category that is not among the predefined set of bug categories provided in the prompt (e.g., documentation issue, memory issue).
- (5) *Additional Text:* The LLM's response includes an additional text block alongside the bug category, which may appear before, after, or mixed within the category label itself.
- (6) *LLM Out-of-Context:* The response fills the entire available context of the LLM, making it incomplete or inconclusive for determining the bug's category. This behavior is most often observed in prompt types that simulate multiple experts to determine the category for the bug report. One root cause is that the simulated experts fail to reach a consensus within the available context. In another case, we observed that the response keeps repeating the same answer for some unknown reason until it fills the available context of model.
- (7) *Consensus Blocks:* In prompt types designed for expert discussions or exploring multiple reasoning paths (ITER, TOT, MAD, SELF), the bug report category is determined within a consensus block that summarizes the final category. For such cases, the consensus block typically provides the determined category in the expected format. We have also observed instances where this consensus block exhibits output formats similar to those seen in regular responses (the ones that does not simulate expert discussions or multiple reasoning paths), including wrapped categories, additional text, multiple categories, and out-of-dictionary category labels. Additionally, this consensus block may lack a definitive conclusion or be missing from the response, making it difficult to extract a clear category.
- (8) *Category Walkthroughs:* The LLM's response evaluates each category, often explaining whether the given bug report fits each category and providing reasons for why it matches or does not match. However, the walkthrough may lack a definitive conclusion or be incomplete, making it difficult to extract a clear category.

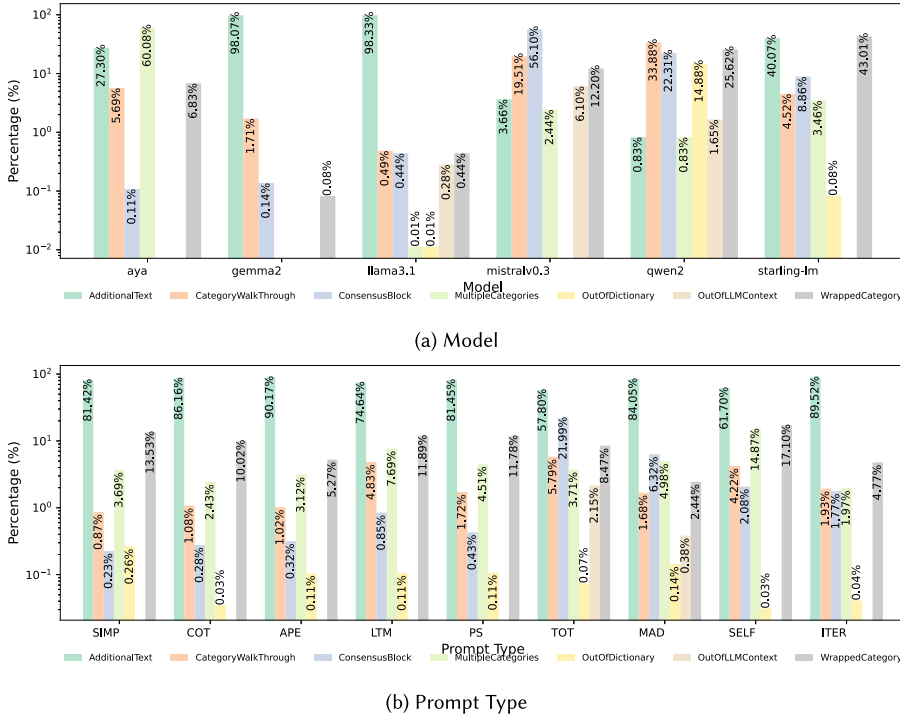


Fig. 2. Distribution of common output pattern deviations.

We observe that the simple regex rule (Equation (1)) fails to capture bug categories when there are minor variations in the generated output. Patterns such as wrapped categories and additional text can be easily covered developing more flexible regex rules. For the prompt types that are designed for expert discussions or exploring multiple reasoning paths, it would be necessary to develop custom regex rules to search for the consensus/final decision in the output of the LLMs.

To better understand the distribution of common output deviations, we analyzed how these deviations occur across different models and prompt types, quantifying their frequency for clearer comparisons. In this analysis, we calculated the percentage of each output pattern deviation relative to the total output for each model and prompt type, allowing for a clearer comparison of the distribution of these deviations. Figure 2 presents the distribution of these common output pattern deviations, first by model and then by prompt type.

As shown in Figure 2(a), the distribution of common output pattern deviations across models reveals significant variations, with some models exhibiting a higher frequency of deviations than others. Gemma2 has minimal deviations, with additional text being the most common issue, indicating relatively consistent performance. Llama3.1 shows similar trends but with a bit more variation, including issues like wrapped category and out of LLM Context. On the other hand, Mistralv0.3 and Qwen2 exhibit more complex issues, such as consensus blocks and multiple categories, suggesting more varied and potentially less stable outputs. Starling-LM shows high occurrences of wrapped category and consensus blocks, indicating that its outputs are more prone to formatting inconsistencies. This data highlights that models like Mistralv0.3, Qwen2, and Starling-LM face a broader range of deviations, while Gemma2 and Llama3.1 exhibit more focused, though still significant, formatting issues.

Figure 2(b), on the other hand, illustrates the distribution of these deviations when categorized by prompt type. The most common deviation across all prompt types is additional text, followed by wrapped category. More complex prompt types like ITER, LTM, and SELF are associated with higher occurrences of consensus blocks and multiple categories, suggesting that these prompt types involve more intricate reasoning or reasoning pathways, which can introduce additional complexities in the output format. In contrast, simpler prompt types such as APE and SIMP exhibit fewer of these deviations but still struggle with formatting issues like additional text and wrapped categories.

Finding 3: The extraction of labels using a simple regex encounter failures when the model provides valid answers in non-standard formats. This underscores the need for developing enhanced regex rules to effectively address the variability in output formats observed across different prompt types and improve category extraction.

Finding 4: Tailored approaches are necessary to improve consistency in output formats and category predictions. Our manual analysis reveals that models frequently disregard the output formatting instructions, adding extra information beyond the category labels. Instructing models to present the extra information in a structured format could help improve output consistency and adherence to output instructions.

RQ1-Summary: The generated outputs across LLMs and prompt types exhibit significant variability in adherence to instructions, with different models demonstrating inconsistent compliance with specified output guidelines. Both the prompt type and the chosen model influence how effectively LLM outputs produce usable categories, resulting in variability in alignment with predefined categories. Our examination reveal that some discarded outputs contain patterns that can yield usable labels with further refinement in extraction methods. Additionally, we observe models often generate extra information beyond the category labels and disregard the output formatting instructions. These findings suggest that with improved extraction methods, some discarded outputs could yield usable labels, while models often ignore formatting instructions by adding extra information, highlighting the need for better output formatting guidelines.

4.2 RQ2: To What Extent Can the Output Format of LLMs Be Controlled Using Prompt Engineering and Lightweight Post-Processing?

4.2.1 Impact of Prompt Engineering on Model Adherence to Instructions.

Approach: We utilize prompt engineering, which involves refining input prompts to enhance model adherence to instructions and improve the consistency of generated responses. Our proposed approach focuses on refining the output format section of the prompt templates, defining distinct output instruction blocks (Process, Explanation, and Category) to ensure that the responses generated by the models are organized and coherent. By doing so, we postulate that this structured output format will prompt models to clearly articulate their reasoning and processing steps to the corresponding output blocks defined in the format, thereby improving verification of prompt adherence and enhancing the consistency of generated responses.

The following details the specific function of each block:

- *Process (P):* This block details the steps models use to generate the bug category, where the prompt type dictates the model to describe how these steps are executed.
- *Explanation (E):* This block provides the reasoning behind why the chosen category is appropriate, the prompt instructs the models to justify their choice.
- *Category (C):* This block is part of the original template and contains only the name of the predicted bug category, which represents the primary outcome of our task.

Table 8. Number of Discarded Responses for Various Language Models across Different Prompt Types under Considered Output Block Configurations

	PEC	EC	PC	C	PEC	EC	PC	C	PEC	EC	PC	C
	Aya				Gemma2				Llama3.1			
SIMP	1	3	1	49	0	0	272	982	0	1	0	1,018
COT	3	2	0	202	0	0	30	947	0	1	0	1,022
APE	1	3	0	137	0	0	9	908	5	4	0	1,024
LTM	3	3	0	3	0	0	3	812	2	35	0	1,024
PS	1	9	3	33	0	0	26	986	1	39	0	1,024
TOT	1	3	2	1	0	120	1	701	10	0	1	640
MAD	0	5	7	41	0	0	0	739	6	1	0	954
	9	1	7	4	0	0	0	781	3	2	0	1,020
ITER	4	2	9	31	0	0	0	924	4	9	2	1,023
Total	23	31	29	501	0	120	341	7,780	31	92	3	8,749
	Mistralv0.3				Qwen2				Starling-LM			
SIMP	3	0	0	0	0	0	0	0	30	1	13	13
COT	0	0	0	6	0	1	3	0	323	12	118	28
APE	0	0	0	0	1	0	1	0	282	19	121	45
LTM	0	0	1	0	0	0	1	0	294	7	96	44
PS	0	0	0	0	0	0	2	0	224	3	47	44
TOT	1	0	11	0	0	0	12	0	426	260	298	199
MAD	1	0	51	0	1	0	0	0	230	129	49	167
SELF	0	0	1	0	4	0	6	0	125	26	48	77
ITER	1	0	19	0	2	0	9	0	257	128	193	66
Total	6	0	83	6	8	1	34	0	2,191	585	983	683

Table 4 presents the specific output instructions given to the models, organized by the various prompt types utilized in our study. To examine the impact of these instructions, we conduct an inverted ablation experiment in which we systematically add additional output formatting instructions to the prompt template and evaluate their effects across models and prompt types. Starting from the Category (C) block alone, we first incorporate the Explanation (E) block to create the EC configuration, follows by the addition of the Process (P) block to form the PC configuration, and finally, we include both Explanation and Process blocks to achieve the PEC configuration. For each configuration, the LLM's response must include all specified blocks to be considered compliant; otherwise, it is discarded. For instance, in the PEC configuration, the response must contain Process (P), Explanation (E), and Category (C) blocks. If any block is missing, the response is marked as non-compliant and discarded. The same principle applies to other configurations: in the EC configuration, both Explanation (E) and Category (C) are required, while the PC configuration mandates the inclusion of Process (P) and Category (C). Similar to RQ1, where we use regular expression 1 to extract the category block from the models' output, we develop two additional regular expressions to capture the Process and Explanation blocks. The only difference in these regular expressions is the name of the block being extracted.

Results: Table 8 shows the number of discarded responses for various language models across different prompt types under the PEC, EC, PC, and C configurations. For models such as Aya, Gemma2, and Llama3.1, we observe significant improvements across all prompt types when PE, E, or P is added to the baseline C, compared to using C alone. For instance, Gemma-2 achieves perfect adherence (0 discards) with the PEC configuration, a dramatic improvement from its 7,780 discards

under C. Llama3.1 similarly sees its discards drop significantly, reaching a low of 3 with the PC configuration.

This positive trend, however, does not hold universally. For Mistralv0.3, Qwen2, and Starling-LM (representing half the models tested), adding these instructional elements does not consistently improve adherence, and in several instances, degrades performance relative to the baseline C. Notably, Qwen2, which exhibits perfect adherence with the baseline C, performs worse under all conditions with added instructions (EC, PC, PEC). Starling-LM also struggles, showing the highest number of discarded responses (2,191) under the most complex PEC condition, significantly worse than its baseline performance (683 discards). Mistralv0.3 shows specific sensitivity; while it improves with EC (0 discards), adding Process instructions (PC) is detrimental (83 discards), particularly for prompt types designed for expert discussions or exploring multiple reasoning paths (ITER, TOT, MAD, SELF), where these discards primarily occur.

A closer analysis of Starling-LM's responses reveals that the model often structures its outputs using enumeration (e.g., "(1) Process, (2) Explanation, (3) Categories") rather than the specified markdown headers (#) to separate blocks, deviating from the expected formatting. Our evaluation relies on regex-based parsing to measure the adherence of the format, which fails to recognize this alternative structure, leading to a higher number of discarded responses. Although these responses may still contain the required information, they do not strictly conform to the specified formatting, causing them to be classified as non-compliant. This highlights a key limitation: rigid structural parsing adherence assessments can misrepresent a model's ability to follow instructions when alternative formatting conventions are used.

Finding 5: Instructing models to present extra information beyond category labels has mixed effects on adherence to the formatting, depending on the model and potentially the complexity of the underlying task. While Gemma2 achieves perfect adherence under PEC, others (e.g., Qwen2, Starling-LM) exhibit an increase in discarded responses, indicating that additional instructions do not universally enhance adherence. In the case of Starling-LM, the model struggles with adherence, ignoring direct formatting instructions, and consistently generating outputs with varied formats, while Mistralv0.3 shows sensitivity related to both the added instruction (PC) and the nature of the prompt type. Model-specific tendencies in structuring outputs play a crucial role in adherence rates. Future work could explore refining prompt instructions to account for these variations, as well as developing evaluation methods that are more robust to diverse formatting styles while still ensuring compliance with output requirements.

4.2.2 Impact of Prompt Engineering and Post-Processing on Adherence to Predefined Bug Categories.

Approach: In this section, we focus on two main objectives. First, we aim to determine whether the prompt engineering introduced in the previous section effectively produces outputs that align with the predefined set of bug categories. Second, we explore whether we can enhance the extraction of categories from the model's outputs through post-processing.

To achieve the second objective, we develop extended regular expressions to better capture variations in category output formats. The initial regex rule (Equation (1)) follows a strict exact-match approach, which fails to extract bug categories when minor variations appear in the generated output. As discussed in Section 4.1.3, we observed patterns such as wrapped categories, additional explanatory text, and alternative formatting styles that led to mismatches. To address these issues, the extended regex rules build upon the original formulation by incorporating more flexible matching patterns that account for these variations. As outlined in Algorithm 1, the process begins by extracting a potential category from the response using the initial regex pattern (Equation (1)). If an exact match to the predefined categories is not found, the algorithm checks for variations such

Algorithm 1: Extended Regex-Based Bug Category Extraction**Require:** *response* (text output from LLM)**Ensure:** *extracted_category* (a valid bug category label or *None*)

```

1: category_candidate  $\leftarrow$  ApplyRegex(response)                                 $\triangleright$  Apply initial regex extraction
2: if category_candidate  $\in$  predefined_categories then
3:   return category_candidate
4: end if
5: category_candidate  $\leftarrow$  ExtractWithinQuotes(response)                       $\triangleright$  Handle quotation variations
6: if category_candidate  $\in$  predefined_categories then
7:   return category_candidate
8: end if
9: category_candidate  $\leftarrow$  ExtractWithAdditionalText(response, predefined_categories)  $\triangleright$  Handle extra text
10: if category_candidate  $\neq$  None then
11:   return category_candidate
12: end if
13: category_candidate  $\leftarrow$  ExtractFromConsensusBlock(response)                 $\triangleright$  Check consensus block
14: if category_candidate  $\in$  predefined_categories then
15:   return category_candidate
16: end if
17: return None                                                                 $\triangleright$  No valid category found

```

as categories wrapped in quotation marks, categories mixed with additional text, and categories identified within consensus blocks. This multi-step process ensures that bug categories are captured despite formatting inconsistencies. Similar to RQ1, we assess LLM adherence to expected bug report categories by comparing generated labels to the predefined set, applying exact-match criteria to discard responses that do not match or list multiple categories.

Results: Prompt Engineering Only. Table 9 shows the number of discarded responses when using simple regex for bug category extraction, highlighting the effectiveness of different configurations (PEC, EC, PC, C). We observe a significant reduction in discarded responses with the EC configuration compared to the C configuration for most models. Aya shows the largest improvement, reducing discarded responses by 90% (from 1,246 to 125). Gemma2 follows with a 78% reduction (from 37 to 8), while Llama3 and Mistral reduce by 59% (from 136 to 56) and 76% (from 421 to 102), respectively. Qwen2 also sees a 57% reduction (from 106 to 46). While most models perform best with the EC configuration, Starling-LM shows a 42% improvement (from 4,101 to 2,374) with the PEC configuration, while EC and PC configurations increase discarded responses. In contrast, models like Llama3 and Qwen2 show higher discard rates with the PEC configuration, while Mistral performs worse with the PC configuration, highlighting the varying impacts of these configurations on each model. We observe no consistent trends for prompt types, with performance varying across models.

Finding 6: Prompt engineering impacts model alignment with predefined bug report categories, but its effectiveness is highly model-dependent. While certain configurations yield substantial improvements for specific models (e.g., the EC configuration reduced discarded responses by 57–90% for Aya, Gemma2, Mistral, Llama3, and Qwen2), the same or similar configurations can be ineffective or even detrimental for others (e.g., the PEC configuration increased discard rates for Llama3 and Qwen2, while the EC configuration worsened performance for Starling-LM, which uniquely benefited from PEC). This variability highlights that while prompt engineering can be an effective strategy to enhance alignment, achieving optimal results requires tailoring configurations to individual models, rather than applying a single strategy universally.

Table 9. Number of Discarded Responses under Different Configurations (PEC, EC, PC, C)
Using Simple Regex to Assess Alignment with Predefined Bug Report Categories

	PEC	EC	PC	C	PEC	EC	PC	C	PEC	EC	PC	C
	Aya				Gemma2				Llama3.1			
SIMP	0	13	14	114	0	1	1	0	15	4	6	2
COT	8	12	8	80	0	0	0	0	19	4	3	1
APE	9	12	13	102	2	1	0	1	10	8	7	0
LTM	15	13	22	266	1	0	0	6	20	5	21	0
PS	30	11	62	143	2	1	0	0	24	6	26	0
TOT	48	12	97	22	6	0	5	7	60	3	4	104
MAD	48	9	53	10	6	3	16	20	63	9	7	28
SELF	24	38	50	496	2	1	2	3	14	7	15	1
ITER	53	5	62	13	1	1	2	0	14	10	11	0
Total	235	125	381	1,246	20	8	26	37	239	56	100	136
	Mistralv0.3				Qwen2				Starling-LM			
SIMP	5	4	7	0	112	2	9	7	240	489	982	450
COT	17	7	24	0	168	4	56	1	316	543	609	580
APE	18	11	25	0	164	5	116	9	322	549	592	603
LTM	14	17	77	1	215	8	89	11	399	523	637	545
PS	8	8	45	1	179	2	25	9	338	483	665	521
TOT	87	12	92	352	18	7	10	38	70	245	225	144
MAD	105	14	116	11	18	4	13	1	114	495	332	216
SELF	36	15	109	56	138	5	112	29	480	543	813	702
ITER	54	14	67	0	4	9	5	1	95	307	281	340
Total	344	102	562	421	1,016	46	435	106	2,374	4,177	5,136	4,101

Results: Prompt Engineering with Post-Processing. Table 10 presents the number of responses discarded when extended regex rules are applied for extraction (absolute values) alongside the change compared to using simple regex (Table 9 and Table 10 values in parentheses). The extended regex rules build on the simple regex rule (Equation (1)) by incorporating rules to handle wrapped category, additional text, and consensus block patterns identified in the previous RQ.

As clearly demonstrated by the overwhelmingly negative values in parentheses in Table 10, the application of extended regex rules as a post-processing step results in a significant reduction in the number of discarded LLM responses across almost all models, prompt types, and configurations compared to using simple regex (Table 9). The total number of discarded responses decreases by 85% overall (from 21,429 to 3,325). For example, Starling-LM shows total discard reductions ranging from −2,259 (PEC) to −4,873 (PC). This highlights the clear overall effectiveness of the extended regex post-processing method itself when applied to the model outputs.

However, examining the results within Table 10 reveals complexities regarding specific prompt instructions. Notably, adding the Process instruction (“P”) compared to the baseline (“C”) resulted in an increase in discarded responses for five out of six models. We attribute this C→PC trend primarily to the increased complexity introduced by requiring the model to generate explicit processing steps within its response, which can negatively impact parsing success by our regex method. This interpretation is supported by the relative performance hierarchy observed within Table 10 for several models (Aya, Mistral, Qwen2, Starling-LM), where EC generally yields fewer discards than PEC, which in turn yields fewer discards than PC (EC < PEC < PC). This suggests “E” often aids regex parsing more effectively than “P.” (Llama3.1 presents a notable exception [PC < PEC < EC], underscoring model-specific interactions).

Table 10. Number of Discarded Responses under Different Configurations (PEC, EC, PC, C) Using Extended Regex Extraction to Assess Alignment with Predefined Bug Categories

	PEC	EC	PC	C	PEC	EC	PC	C	PEC	EC	PC	C
	Aya				Gemma2				Llama3.1			
SIMP	0 (-)	13 (-)	11 (-3)	98 (-16)	0 (-)	1 (-)	1 (-)	0 (-)	1 (-14)	1 (-3)	0 (-6)	0 (-2)
COT	6 (-2)	11 (-1)	6 (-2)	70 (-10)	0 (-)	0 (-)	0 (-)	0 (-)	2 (-17)	1 (-3)	1 (-2)	0 (-1)
APE	7 (-2)	12 (-)	11 (-2)	88 (-14)	2 (-)	1 (-)	0 (-)	0 (-1)	1 (-9)	4 (-4)	1 (-6)	0 (-)
LTM	12 (-3)	13 (-)	17 (-5)	219 (-47)	1 (-)	0 (-)	0 (-)	0 (-6)	1 (-19)	0 (-5)	1 (-20)	0 (-)
PS	26 (-4)	11 (-)	26 (-36)	126 (-17)	2 (-)	1 (-)	0 (-)	0 (-)	1 (-23)	3 (-3)	1 (-25)	0 (-)
TOT	34 (-14)	11 (-1)	69 (-28)	15 (-7)	4 (-2)	0 (-)	4 (-1)	4 (-3)	6 (-54)	2 (-1)	1 (-3)	5 (-99)
MAD	41 (-7)	8 (-1)	43 (-10)	10 (-)	6 (-)	3 (-)	12 (-4)	12 (-8)	3 (-60)	6 (-3)	0 (-7)	3 (-25)
SELF	20 (-4)	38 (-)	44 (-6)	471 (-25)	2 (-)	1 (-)	2 (-)	0 (-3)	3 (-11)	2 (-5)	2 (-13)	0 (-1)
ITER	34 (-19)	5 (-)	49 (-13)	13 (-)	1 (-)	1 (-)	2 (-)	0 (-)	5 (-9)	7 (-3)	6 (-5)	0 (-)
Total	180 (-55)	122 (-3)	276 (-105)	1,110 (-136)	18 (-2)	8 (-)	21 (-5)	16 (-21)	23 (-216)	26 (-30)	13 (-87)	8 (-128)
	Mistralv0.3				Qwen2				Starling-LM			
SIMP	2 (-3)	3 (-1)	3 (-4)	0 (-)	1 (-111)	1 (-1)	0 (-9)	7 (-)	3 (-237)	2 (-487)	4 (-978)	1 (-449)
COT	8 (-9)	0 (-7)	12 (-12)	0 (-)	4 (-164)	1 (-3)	5 (-51)	1 (-)	2 (-314)	5 (-538)	17 (-592)	4 (-576)
APE	5 (-13)	5 (-6)	15 (-10)	0 (-)	4 (-160)	0 (-5)	11 (-105)	5 (-4)	3 (-319)	2 (-547)	5 (-587)	3 (-600)
LTM	4 (-10)	5 (-12)	14 (-63)	1 (-)	0 (-215)	2 (-6)	9 (-80)	3 (-8)	0 (-399)	4 (-519)	19 (-618)	2 (-543)
PS	4 (-4)	3 (-5)	2 (-43)	1 (-)	5 (-174)	1 (-1)	2 (-23)	4 (-5)	0 (-338)	2 (-481)	14 (-651)	1 (-520)
TOT	56 (-31)	9 (-3)	69 (-23)	220 (-132)	3 (-15)	1 (-6)	3 (-7)	9 (-29)	38 (-32)	12 (-233)	63 (-162)	36 (-108)
MAD	63 (-42)	6 (-8)	83 (-33)	8 (-3)	2 (-16)	3 (-1)	7 (-6)	1 (-)	45 (-69)	7 (-488)	79 (-253)	70 (-146)
SELF	24 (-12)	8 (-7)	68 (-41)	43 (-13)	6 (-132)	2 (-3)	16 (-96)	3 (-26)	12 (-468)	3 (-540)	36 (-777)	10 (-692)
ITER	24 (-30)	6 (-8)	25 (-42)	0 (-)	1 (-3)	6 (-3)	3 (-2)	1 (-)	12 (-83)	12 (-295)	26 (-255)	20 (-320)
Total	190 (-154)	45 (-57)	291 (-271)	273 (-148)	26 (-990)	17 (-29)	56 (-379)	34 (-72)	115 (-2,259)	49 (-4,128)	263 (-4,873)	147 (-3,954)

The table includes both the raw discarded response counts and the improvement (difference) compared to Table 9, highlighting the overall reduction in discarded responses with extended regex post-processing.

While the discard counts in Table 10 suggest challenges with “P” for regex parsing, focusing solely on this metric overlooks benefits related to which specific LLM responses can be successfully processed. To quantify this, we analyzed the ability of configurations with added “P” or “E” instructions to successfully extract categories from LLM responses corresponding to bug reports where the respective baseline configuration had failed. This assesses the change in category extraction coverage across different sets of bug reports.

Specifically, for each relevant transition pair (e.g., C vs. PC), we calculated:

- *Gained Unique*: The number of LLM responses (each corresponding to a unique bug report) from which the compared configuration (e.g., PC) successfully extracted a category, but the baseline configuration (e.g., C) failed. This measures the expansion of coverage to new items for that specific comparison.
- *Lost Unique*: The number of LLM responses from which the baseline configuration (e.g., C) succeeded, but the compared configuration (e.g., PC) failed. This measures the loss of coverage on previously successful items for that specific comparison.
- *Net Change*: The difference between gained unique and lost unique (“Gained–Lost”). A positive value indicates the compared configuration covered more unique items than it lost relative to the baseline; a negative value indicates it lost more than it gained.

Table 11 summarizes this analysis, showing the net change (and the underlying gained/lost counts) for each transition across all models and prompt types.

To understand the overall trends across all 108 comparisons for adding “P” and 108 for adding “E,” we calculated aggregate frequencies, summarized in Table 12. The percentages in Table 12 represent the proportion of comparisons exhibiting a certain outcome:

- “% Gained Unique Categories” indicates the percentage of comparisons where the augmented configuration had “Gained Unique > 0.”
- “% Lost Unique Categories” indicates the percentage where “Lost Unique > 0.”

Table 11. Pairwise Analysis of Unique Category Extraction Gains and Losses

Prompt Type	Aya				Gemma2				Llama3.1			
	C→PC	C→EC	EC→PEC	PC→PEC	C→PC	C→EC	EC→PEC	PC→PEC	C→PC	C→EC	EC→PEC	PC→PEC
SIMP	+86 (97/11)	+84 (96/12)	+13 (13/0)	+11 (11/0)	-1 (0/1)	-1 (0/1)	+1 (1/0)	+1 (1/0)	0 (0/0)	-1 (0/1)	0 (1/1)	-1 (0/1)
COT	+64 (69/5)	+58 (69/11)	+5 (10/5)	-1 (6/7)	0 (0/0)	0 (0/0)	0 (0/0)	0 (0/0)	0 (1/1)	0 (1/1)	-1 (1/2)	-1 (0/1)
APE	+77 (88/11)	+75 (87/12)	+5 (12/7)	+3 (11/8)	0 (0/0)	-1 (0/1)	-1 (0/1)	-2 (0/2)	0 (1/1)	-3 (1/4)	+2 (4/2)	-1 (1/2)
LTM	+203 (214/11)	+206 (213/7)	+1 (12/11)	+4 (17/13)	0 (0/0)	0 (0/0)	-1 (0/1)	-1 (0/1)	-1 (0/1)	0 (0/0)	-1 (0/1)	0 (1/1)
PS	+98 (117/19)	+114 (122/8)	-14 (10/24)	+2 (26/24)	0 (0/0)	-1 (0/1)	-1 (0/1)	-2 (0/2)	0 (1/1)	-2 (1/3)	+2 (3/1)	0 (1/1)
TOT	-54 (14/68)	+3 (15/12)	-23 (9/32)	+34 (67/33)	+634 (635/1)	+527 (542/15)	+107 (111/4)	0 (4/4)	+42 (44/2)	+42 (44/2)	-7 (2/9)	-7 (1/8)
MAD	-33 (10/43)	+2 (11/9)	-32 (9/41)	+3 (40/37)	0 (12/12)	+9 (12/3)	-3 (2/5)	+6 (11/5)	+47 (47/0)	+41 (47/6)	+3 (6/3)	-3 (0/3)
SELF	+421 (444/23)	+432 (450/18)	+15 (38/23)	+26 (45/19)	-2 (0/2)	-1 (0/1)	-1 (0/1)	0 (1/1)	-1 (1/2)	-1 (1/2)	-1 (2/3)	-1 (2/3)
ITER	-36 (11/47)	+7 (13/6)	-28 (6/34)	+15 (46/31)	-2 (0/2)	-1 (0/1)	0 (0/0)	+1 (2/1)	-7 (0/7)	-7 (0/7)	+2 (7/5)	+2 (7/5)
Total net	+770	+858	-54	+92	+629	+524	+98	+4	+79	+69	-10	-10
(Gain/loss)	(1,044/237)	(1,077/219)	(105/159)	(218/126)	(649/20)	(554/30)	(116/18)	(10/6)	(96/17)	(96/27)	(30/40)	(14/24)
Prompt Type	Mistralv0.3				Qwen2				Starling-LM			
	C→PC	C→EC	EC→PEC	PC→PEC	C→PC	C→EC	EC→PEC	PC→PEC	C→PC	C→EC	EC→PEC	PC→PEC
SIMP	-3 (0/3)	-3 (0/3)	0 (3/3)	0 (3/3)	+7 (7/0)	+6 (7/1)	0 (1/1)	-1 (0/1)	-4 (1/5)	-1 (1/2)	-1 (2/3)	+2 (5/3)
COT	-12 (0/12)	0 (0/0)	-8 (0/8)	+4 (12/8)	-4 (1/5)	0 (1/1)	-3 (1/4)	+1 (5/4)	-13 (4/17)	-1 (4/5)	+2 (5/3)	+14 (17/3)
APE	-15 (0/15)	-5 (0/5)	0 (5/5)	+10 (15/5)	-6 (5/11)	+5 (5/0)	-4 (0/4)	+7 (11/4)	-2 (4/6)	+2 (4/2)	-3 (2/5)	+1 (6/5)
LTM	-13 (1/14)	-4 (1/5)	+1 (5/4)	+10 (14/4)	-6 (2/8)	+1 (3/2)	+2 (2/0)	+9 (9/0)	-17 (3/20)	-1 (3/4)	-2 (3/5)	+14 (20/6)
PS	-1 (1/2)	-2 (1/3)	-1 (3/4)	-2 (2/4)	+2 (4/2)	+3 (4/1)	-4 (1/5)	-3 (2/5)	-10 (4/14)	+2 (4/2)	+2 (2/0)	+14 (14/0)
TOT	+146 (205/59)	+211 (219/8)	-48 (8/56)	+17 (68/51)	+5 (9/4)	+8 (9/1)	-2 (1/3)	+1 (4/3)	-31 (47/78)	+35 (53/18)	-44 (20/64)	+22 (77/55)
ITER	-29 (0/29)	-6 (0/6)	-19 (5/24)	+4 (28/24)	-3 (1/4)	-5 (1/6)	+5 (6/1)	+3 (4/1)	-25 (28/53)	+13 (30/17)	+2 (16/14)	+40 (55/15)
MAD	-80 (7/87)	+2 (8/6)	-57 (6/63)	+25 (82/57)	-6 (1/7)	-2 (1/3)	+1 (3/2)	+5 (7/2)	+28 (104/76)	+106 (111/5)	-54 (7/61)	+24 (73/49)
SELF	-25 (37/62)	+35 (43/8)	-16 (8/24)	+44 (65/21)	-13 (3/16)	+1 (3/2)	-4 (2/6)	+10 (15/5)	-15 (20/35)	+19 (23/4)	-9 (3/12)	+25 (35/10)
Total net	+14	+229	-148	+108	-22	+20	-9	+35	-70	+174	-111	+176
(Gain/loss)	(251/237)	(271/42)	(43/191)	(294/186)	(35/57)	(37/17)	(18/27)	(57/22)	(223/293)	(265/91)	(70/181)	(282/106)

Totals represent the sum of net changes and the sum of gains/losses across all prompt types for that model/transition. Cell background color indicates the net change direction (Green = positive net change, Red = negative net change).

Table 12. Summary of Unique Category Extraction Coverage Changes

Metric	Added “P” (C->PC, EC->PEC)	Added “E” (C->EC, PC->PEC)
Total comparisons	108	108
% Gained unique categories	78.7% (85)	79.6% (86)
% Lost unique categories	88.0% (95)	90.7% (98)
% Resulting in net gain	29.6% (32)	58.3% (63)
% Resulting in net loss	56.5% (61)	30.6% (33)
% Resulting in no net change	13.9% (15)	11.1% (12)

Numbers in parentheses indicate absolute counts.

- “% Resulting in Net Gain” indicates the percentage where “Net Change > 0” (“Gained > Lost”).
- “% Resulting in Net Loss” indicates the percentage where “Net Change < 0” (“Lost > Gained”).

Note that a single comparison can contribute to both “% Gained” and “% Lost” if it has both types of changes. Based on these analyses:

- *Impact of Adding “P”*: Adding “P” frequently enabled successful category extraction from LLM responses corresponding to previously uncovered bug reports (gained unique > 0), occurring in 78.7% of the 108 relevant comparisons (Table 12). Notably, for the specific C->PC comparison, even when the total extraction count decreased (in 31 out of 54 cases), adding “P” still succeeded on unique LLM responses in 22 of those instances (71%). However, adding “P” also frequently resulted in failing on responses previously covered by the baseline (lost unique > 0) (occurring in 88.0% of comparisons, Table 12). The prevalence of losses often outweighed the gains, resulting in a net loss (net change < 0) much more often than a net gain (56.5% net loss vs. 29.6% net gain overall, Table 12, also visible by the frequent negative values in Table 11).
- *Impact of Adding “E”*: Adding “E” similarly enabled success on unique LLM responses (gained unique > 0) in 79.6% of its 108 comparisons (Table 12). While adding “E” also involved frequently losing coverage on baseline-successful responses (lost unique > 0) (in 90.7%), it

resulted in a net gain (net change > 0) significantly more often than a net loss (58.3% net gain vs. 30.6% net loss overall, Table 12), reflected in the generally positive net changes shown in Table 11.

This detailed analysis (Table 11, summarized in Table 12) indicates that while both “P” and “E” instructions often broaden coverage by enabling successful category extraction on different sets of LLM responses, “E” achieves a more favorable overall balance, resulting more often in a net expansion of uniquely covered responses. “P,” despite unlocking unique cases with similar frequency, more often results in a net reduction, likely linked to the parsing challenges reflected in the discard counts (Table 10). We also observe (from Table 11) that prompt types instructing multi-step reasoning (TOT, MAD, SELF, ITER) combined with Process blocks (PC, PEC) particularly challenged models like Aya, Mistral, and Starling-LM, contributing to the observed patterns.

Finding 7: Combining prompt engineering (adding “E” or “P” instructions) with extended regex post-processing significantly improves overall output consistency compared to simple regex. Both “E” and “P” instructions broaden coverage by enabling successful category extraction from LLM responses for bug reports not covered by baseline configurations (79% frequency). However, adding “P” introduces complexity that more frequently increases regex parsing failures (discards) and leads to a net reduction in the set of uniquely covered responses compared to adding “E,” which achieves a net gain in unique coverage more often (Tables 11 and 12).

RQ2-Summary: LLMs do not always align with instructions, and this makes it challenging to extract relevant information from their outputs. Our examination reveals that the outputs can be controlled can significantly enhance output consistency across different models and prompt types through a combination of prompt engineering and lightweight post-processing. Prompt engineering, through refined output formatting instructions (like adding Explanation “E” or Process “P” blocks), helps ensure models generate category labels. Both “E” and “P” broaden coverage by enabling successful extraction from unique LLM responses missed by simpler configurations (79% frequency). However, they present different tradeoffs regarding complexity and the net change in which responses are successfully processed: “P” often increases parsing complexity (leading to more discards than “E” relative to baselines) and results in a net reduction in uniquely covered responses more frequently, while “E” achieves a net expansion more often, suggesting “E” provides a better overall balance. Lightweight post-processing, using extended regex, is crucial overall and enables the extraction of bug report categories from varied LLM output formats that would otherwise be discarded, yielding substantial improvements over simple regex approaches.

4.3 RQ3: How Do Prompt Engineering Strategies Impact the Performance of LLMs in Categorizing Bug Reports?

4.3.1 Impact of Prompt Engineering Strategies on Model Effectiveness.

Approach: In our first experiment, we investigate the effectiveness of prompt engineering strategies on model performance in categorizing bug reports by comparing two key settings: *Prompt Engineering Only*, which involves incorporating specific directives for reasoning strategies within the instruction sections (prompt types) of the prompts and refining the output formatting instructions of the prompts by defining distinct output instruction blocks (output block configurations); and *Prompt Engineering with Post-Processing*, which combines the prompt engineering strategies with a lightweight post-processing technique that is utilizing extended regular expression parsing to further extract bug report categories from the model’s outputs. We evaluate and compare these settings using **precision, recall, and F1-scores**. Precision is the fraction of LLM responses correctly

assigned that category out of all responses assigned that category by the LLM, reflecting the accuracy of positive predictions. Recall is the fraction of bug reports actually belonging to that category that were correctly identified by the LLM, reflecting the model's ability to find all relevant instances of that category. The F1-score is the harmonic mean of precision and recall (calculated as $2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$), providing a single metric that balances both concerns, which is particularly useful when class distribution may be uneven. To assess overall performance across all categories, we use micro-averaging, which calculates metrics globally by summing true positive, false positive, and false negative counts across all classes. In our single-label classification task, micro-averaged precision, recall, and F1 are equivalent to overall Accuracy (proportion of correctly classified instances). Therefore, we primarily report the overall Micro-F1-score when discussing general performance.

In the second experiment, we investigate how different LLMs respond to various prompt engineering strategies (prompt types and output block configurations) with post-processing to determine their impact on the classification of bug reports. We select the prompt type SIMP and output block configuration C as our baseline to establish a clear benchmark for comparison. The SIMP prompt is chosen because it provides minimal guidance, simply instructing the models to categorize bug reports, which allows us to identify fundamental classification capabilities. Meanwhile, the output block configuration C directs the models to output only the predicted bug categories, avoiding additional output instructions. This analysis enables us to evaluate whether alternative prompt types and output block combinations can enhance model classification performance. We measure this performance using F1-scores, which balance precision and recall, offering a comprehensive metric for evaluating the models' classification effectiveness.

Results: Impact of Post-Processing. Figure 3 presents a comparison of the distributions of F1-scores, recall, and precision for the two settings: Prompt Engineering Only and Prompt Engineering with Post-Processing. In the boxplots for F1-score and recall, the Prompt Engineering with Post-Processing setting shows a narrower interquartile range, indicating improved consistency and stability in model performance. This suggests post-processing enhances bug categorization reliability. Additionally, the Q0 (minimum) values are generally higher in the Post-Processing setting, reflecting better performance even in the least favorable cases. However, the maximum values (Q4) remain similar between the two settings, with only slight improvements in a few models. This suggests that post-processing improves the consistency and reliability of model performance, particularly in handling less favorable cases, but has a limited effect on the best-case performance.

In contrast, the distribution of precision does not show significant differences between the two settings. The consistency in precision across both configurations implies that while the addition of post-processing improves F1-scores and recall, it may not have the same impact on precision. This could suggest that the models maintain a stable rate of true positive classifications relative to false positives, regardless of the output instruction enhancements.

Finding 8: Integrating lightweight post-processing with prompt engineering improves the consistency and effectiveness of F1-score and recall for categorizing bug reports, while having a minimal effect on precision.

Results: Impact of Prompt Types and Output Block Configurations. Figure 4 shows the distribution of model performance across various prompt types and output block configurations. The trend lines in the output blocks (Figure 4(a)) represent the average F1-scores for each model across different output blocks. We calculate the slope values to represent the relationship between the output blocks and the median F1-scores for each model, aiming to examine the impact of adding additional output instructions on classification performance. For the output blocks (Figure 4(a)), the slope values are as follows: Mistralv0.3 has a slope of 0.0016, Qwen2 is 0.0115, Starling-LM is -0.0045 , Aya is

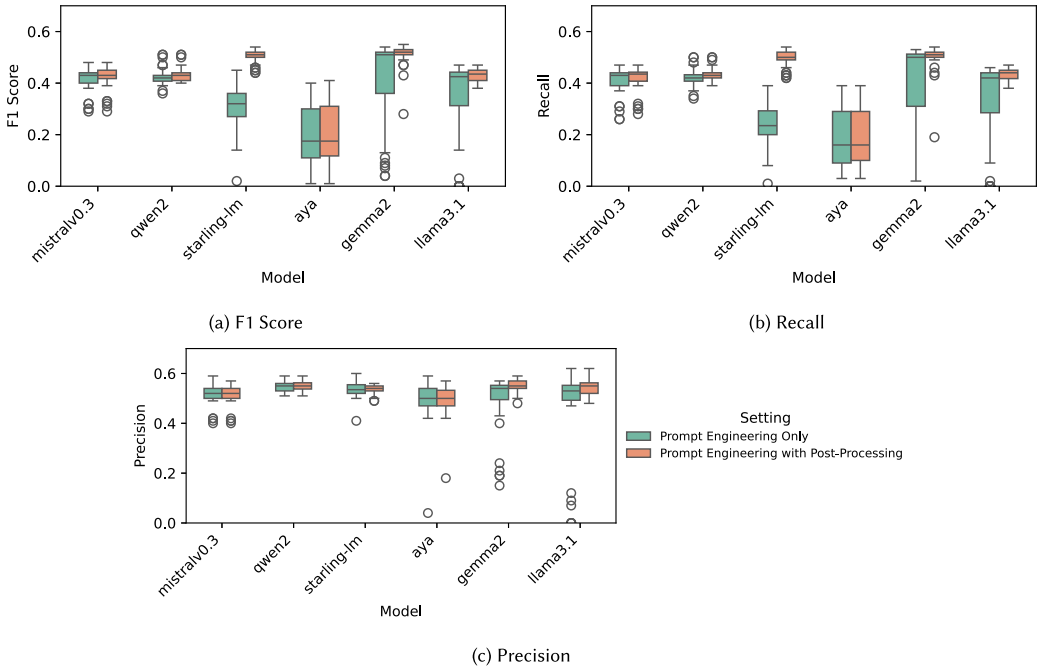


Fig. 3. Impact of post-processing on bug report classification performance.

−0.0722, Gemma2 is 0.0029, and Llama3.1 is −0.0082. For most models, the slopes are near zero, whether positive or negative, indicating that their performance remains largely stable across the different output blocks, with only minor variations in the median F1-scores. This suggests that the choice of output block has a minimal impact on the performance of these models. Qwen2 and Aya are exceptions; Qwen2 (0.0115) shows the most noticeable improvement in F1-scores across output blocks, reflecting a positive trend. This positive trend suggests model performs significantly worse with the additional output instructions (EC, PC, PEC) compared to the baseline configuration C. In contrast, Aya (−0.0722) exhibits a significant performance drop across output blocks, indicating a steeper downward trend. However, this decline is noteworthy for our analysis, as it suggests that the model performs significantly better when using the output blocks with extra instructions compared to the baseline (C).

Figure 4(b) illustrates how sensitive each model's performance is in terms of F1-score to the choice of prompt type. The performance of most models varies significantly depending on the prompt type used. We conduct a one-way ANOVA test to evaluate the impact of different prompt types on the F1-scores of multiple models. In this analysis, the prompt type served as the independent variable, and the F1-score was the dependent variable. The one-way ANOVA tests whether the mean F1-scores significantly differ across the different prompt types for each model. We perform this test separately for each model to assess how prompt type influences classification performance. The results of the ANOVA tests reveal varying levels of impact from prompt types on the performance of different models. For models like Mistral (p-value = 0.00087) and Starling-LM (p-value = 0.00032), the prompt type has a highly significant effect on F1-scores, indicating that adjusting the prompt can substantially influence model performance. Similarly, Gemma2 (p-value = 0.022) and Llama3.1 (p-value = 0.016) also show statistically significant differences in performance across prompt types, suggesting that prompt engineering can improve classification effectiveness. However, for models

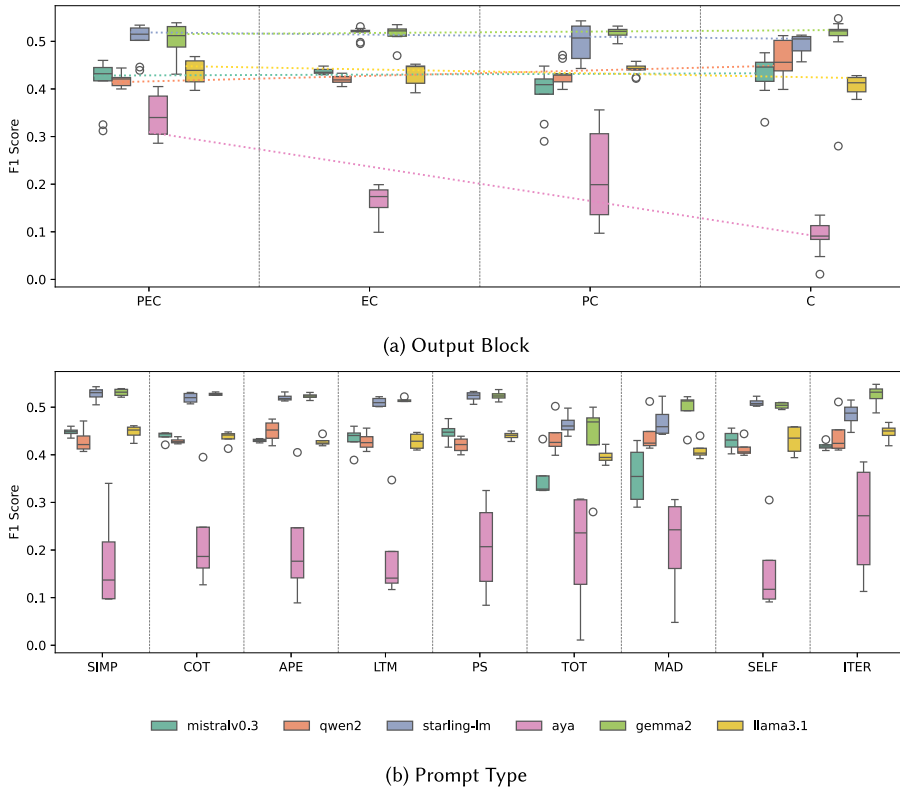


Fig. 4. Impact of prompt types and output block configurations on model performance.

like Aya (p-value = 0.980) and Qwen2 (p-value = 0.816), there is no significant impact from varying the prompt type, implying that their performance is relatively insensitive to prompt modifications. These findings highlight that the effect of prompt type on model performance varies, with some models being more responsive to prompt engineering than others.

Table 13 presents a detailed analysis of model performance variations across various prompt types and output block configurations. In the *prompt type only setting*, the output block is maintained as C (output only category) to isolate the impact of the prompt type on model classification performance. We observe improvements ranging from 1% to 38% over the baseline configuration, SIMP, which is the naive prompt type that simply instructs the models to categorize bug reports. These variations indicate that certain prompt types can significantly enhance model performance, leading to better classification of bug categories. However, certain prompt types, such as TOT and SELF, show notable performance decreases, ranging from 7% to 89% compared to the baseline. This decline underscores that not all prompt types positively impact model performance, highlighting the critical importance of careful prompt selection. The *prompt types combined with output blocks* further improve the models classification performance. Particularly, the Aya model achieves its best F1-score of 0.405 with the APE prompt and PEC configuration, reflecting an impressive 313% improvement over the baseline (SIMP C). This indicates that using APE prompting alongside explicit instructions to detail the process and reasoning behind categorization, significantly enhances classification capabilities. Conversely, prompt types involving multiple perspectives and collaboration (TOT, MAD, SELF, ITER) show mixed results. Table 13 indicates that TOT and MAD perform poorly for most models,

Table 13. Impact of Prompt Types and Output Block Configurations on Model Performance

Model	Rank	Prompt Type Only				Prompt Type with Output Block			
		Prompt Type	Output Block	F1	Imp. %	Prompt Type	Output Block	F1	Imp. %
Aya	Best	LTM	C	0.135	38%	APE	PEC	0.405	313%
	Worst	TOT	C	0.011	−89%	TOT	C	0.011	−89%
	Baseline	SIMP	C	0.098	−	SIMP	C	0.098	−
Gemma2	Best	ITER	C	0.548	2%	ITER	C	0.548	2%
	Worst	TOT	C	0.280	−48%	TOT	C	0.280	−48%
	Baseline	SIMP	C	0.537	−	SIMP	C	0.537	−
Llama3.1	Best	PS	C	0.428	1%	ITER	PEC	0.468	10%
	Worst	TOT	C	0.378	−11%	TOT	C	0.378	−11%
	Baseline	SIMP	C	0.424	−	SIMP	C	0.424	−
Mistralv0.3	Best	PS	C	0.476	6%	PS	C	0.476	6%
	Worst	TOT	C	0.330	−27%	MAD	PC	0.290	−35%
	Baseline	SIMP	C	0.449	−	SIMP	C	0.449	−
Qwen2	Best	MAD	C	0.512	19%	MAD	C	0.512	19%
	Worst	SELF	C	0.399	−7%	TOT	C	0.399	−7%
	Baseline	SIMP	C	0.429	−	SIMP	C	0.429	−
Starling-LM	Best	APE	C	0.513	2%	SIMP	PC	0.543	8%
	Worst	TOT	C	0.457	−10%	TOT	PEC	0.439	−13%
	Baseline	SIMP	C	0.505	−	SIMP	C	0.505	−

possibly due to the complexity of the collaborative structure. However, Qwen2 excels with the MAD prompt, suggesting it handles this structure well. The ITER prompt, specifically designed for bug report categorization, yields the best results for most models, highlighting that a structured collaborative approach focused on the task of categorizing bug reports is more effective than more general or complex collaborative structures.

Finding 9: Leveraging prompt engineering (prompt types and output blocks) can lead to significant improvements, underscoring the critical role of prompt design in optimizing classification tasks. The baseline configuration (SIMP C) serves as an important reference, demonstrating that while simple prompts provide a basic performance level, more specialized prompt types and output structures can unlock substantial gains in specific contexts. This analysis suggests that careful experimentation with different prompt engineering strategies is essential for maximizing classification effectiveness, while also considering the potential drawbacks of suboptimal configurations.

4.3.2 Impact of Prompt Engineering Strategies on Models' Ability to Categorize Different Bug Types.

Approach: To evaluate the impact of prompt engineering strategies on bug report categorization, we focus on the Prompt Engineering with Post-Processing setting, as it shows the best overall performance, as demonstrated earlier. We conduct two distinct experiments for evaluation. The first experiment evaluates model performance across bug report classes, analyzing precision, recall, and F1-scores to assess the models' overall ability to classify all bug types and identify any challenges in specific classes. Due to space constraints, we report only the F1-score, as it provides a balanced measure of precision and recall, offering a clear indicator of classification performance. In the second experiment, we focus on the impact of prompt engineering strategies on the classification effectiveness of different bug report types. Here, we explore how tailored prompt types and output block configurations influence the models' ability to accurately classify bug report categories, comparing the results against a baseline configuration (SIMP C) to assess the performance.

Results: Figure 5 illustrates the per-class F1-score distribution, highlighting model performance across different bug types under the Prompt Engineering with Post-Processing setting. The results

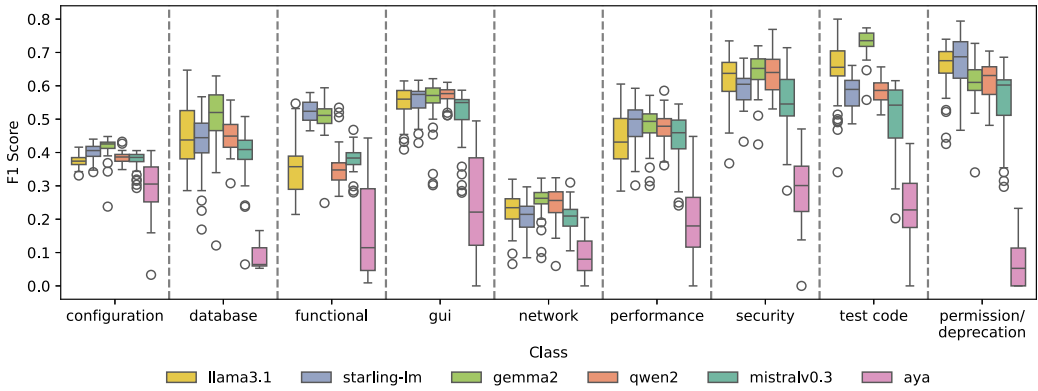


Fig. 5. Distribution model performance per class (F1-score).

highlight several key observations regarding the model performance across different bug types. Models perform relatively well on *security*, *test code*, and *permission/deprecation* issues, with good average F1-scores (mean: 0.5554, 0.5462, and 0.5291, respectively), though moderate variability suggests some inconsistency in classification, indicating room for improvement. While some categories like *configuration* issues show consistent performance (SD: 0.0547), others, such as *functional* and *database* issues, exhibit higher variability (SD: 0.1396 and 0.1608), indicating inconsistent model performance. The Aya model consistently underperforms across all categories, showing particularly low F1-scores even in areas where other models perform better, underscoring its limitations. The *network* issues category poses the greatest challenge, with the lowest mean F1-score (0.2028).

Finding 10: Despite some strong peak performances (e.g., 0.8000 for test issues, 0.7941 for permission/deprecation issues, 0.7692 for security issues), the results suggest that models achieve high performance only in specific instances, and overall, further optimization is needed, especially for challenging categories. These findings point to the need for continued refinement of model approaches, particularly for bug types like network and functional issues, where performance is notably inconsistent, and for addressing the significant weaknesses of the Aya model. Potential strategies to achieve this include prompt engineering, selecting larger models, increasing context size, and applying model fine-tuning. By optimizing these factors, we may be able to enhance the model's ability to handle a broader range of bug types and achieve more consistent and accurate results across the board.

Table 14 provides a detailed breakdown of each model's performance, highlighting the best, worst, and baseline results across various prompt configurations. The findings underscore the significant impact that different prompt types and output configurations have on model performance. In all model-class combinations—except for the Llama3.1 model in the security issues class—the models achieve scores that exceed the baseline prompt's performance (SIMP C). This suggests that the prompt engineering strategies implemented in this study mostly enhance classification effectiveness beyond what is achieved with the baseline prompt. However, in the case of the Llama3.1 model specifically for security issues, the results did not show improvement, indicating that the simple prompting still poses a strong baseline for this particular model and class combination.

In analyzing model performance across 54 model-class pairs, we observe that the TOT, MAD, ITER, and SELF prompt types—which are designed to simulate complex reasoning or collaborative decision-making—consistently result in the lowest performance for 52 pairs. This trend suggests that, for the majority of cases, these prompt types may introduce complexity that hinders effective classification. However, in 21 of model-class pairs, these same prompt types yield the highest

Table 14. Model Performance per Class: Best, Worst, and Baseline Configurations

C	Rank	Aya	Gemma2	Llama3.1	Mistralv0.3	Qwen2	Starling-LM
1	Best	ITER/PEC (0.41)	ITER/EC (0.45)	LTM/PC (0.42)	COT/PC (0.41)	MAD/C (0.43)	APE/PC (0.44)
	Worst	TOT/C (0.03)	TOT/C (0.24)	MAD/PEC (0.33)	MAD/PC (0.29)	TOT/PC (0.35)	ITER/C (0.34)
	Baseline	SIMP/C (0.25)	SIMP/C (0.44)	SIMP/C (0.37)	SIMP/C (0.39)	SIMP/C (0.39)	SIMP/C (0.41)
2	Best	COT/PEC (0.17)	SELF/PEC (0.63)	SIMP/C (0.65)	APE/PEC (0.51)	SELF/PEC (0.56)	APE/C (0.57)
	Worst	LTM/PC (0.05)	TOT/C (0.12)	SELF/PEC (0.29)	TOT/C (0.06)	ITER/PC (0.31)	MAD/PEC (0.17)
	Baseline	SIMP/C (0.06)	SIMP/C (0.62)	SIMP/C (0.65)	SIMP/C (0.35)	SIMP/C (0.44)	SIMP/C (0.54)
3	Best	APE/PEC (0.44)	ITER/PEC (0.59)	ITER/PEC (0.55)	PS/C (0.47)	TOT/C (0.53)	ITER/PEC (0.58)
	Worst	TOT/C (0.01)	TOT/C (0.25)	SELF/C (0.21)	LTM/PC (0.28)	SELF/C (0.27)	TOT/C (0.47)
	Baseline	SIMP/C (0.03)	SIMP/C (0.51)	SIMP/C (0.27)	SIMP/C (0.41)	SIMP/C (0.32)	SIMP/C (0.49)
4	Best	APE/PEC (0.49)	ITER/C (0.62)	SIMP/PC (0.61)	SIMP/PEC (0.59)	COT/C (0.61)	SELF/C (0.62)
	Worst	TOT/C (0.00)	TOT/C (0.30)	ITER/PC (0.41)	TOT/PEC (0.28)	ITER/PEC (0.51)	ITER/PC (0.43)
	Baseline	SIMP/C (0.08)	SIMP/C (0.60)	SIMP/C (0.60)	SIMP/C (0.56)	SIMP/C (0.61)	SIMP/C (0.58)
5	Best	ITER/PC (0.21)	PS/PEC (0.32)	TOT/PC (0.32)	TOT/EC (0.31)	PS/PC (0.32)	LTM/C (0.30)
	Worst	SELF/C (0.00)	TOT/PEC (0.08)	ITER/PEC (0.07)	TOT/PC (0.11)	ITER/PEC (0.06)	ITER/PC (0.08)
	Baseline	SIMP/C (0.05)	SIMP/C (0.25)	SIMP/C (0.31)	SIMP/C (0.23)	SIMP/C (0.26)	SIMP/C (0.22)
6	Best	COT/PEC (0.45)	ITER/PC (0.57)	APE/C (0.61)	LTM/EC (0.55)	LTM/PEC (0.59)	SIMP/PEC (0.59)
	Worst	TOT/C (0.00)	TOT/EC (0.30)	LTM/PC (0.28)	TOT/PC (0.24)	ITER/PEC (0.36)	MAD/PC (0.30)
	Baseline	SIMP/C (0.19)	SIMP/C (0.54)	SIMP/C (0.60)	SIMP/C (0.41)	SIMP/C (0.48)	SIMP/C (0.51)
7	Best	MAD/PC (0.23)	COT/C (0.73)	ITER/EC (0.74)	ITER/C (0.69)	PS/EC (0.70)	SELF/C (0.79)
	Worst	ITER/C (0.00)	TOT/C (0.34)	MAD/PEC (0.43)	TOT/PEC (0.30)	ITER/PC (0.48)	ITER/PC (0.47)
	Baseline	SIMP/C (0.00)	SIMP/C (0.72)	SIMP/C (0.66)	SIMP/C (0.63)	SIMP/C (0.57)	SIMP/C (0.78)
8	Best	ITER/C (0.47)	LTM/PEC (0.72)	SIMP/C (0.73)	SIMP/PEC (0.71)	SIMP/PC (0.77)	TOT/C (0.68)
	Worst	TOT/C (0.00)	TOT/C (0.42)	MAD/PEC (0.37)	TOT/PC (0.29)	TOT/EC (0.53)	MAD/PEC (0.43)
	Baseline	SIMP/C (0.20)	SIMP/C (0.69)	SIMP/C (0.73)	SIMP/C (0.56)	SIMP/C (0.66)	SIMP/C (0.59)
9	Best	PS/PEC (0.43)	PS/PEC (0.77)	SIMP/EC (0.80)	PS/EC (0.62)	SIMP/PC (0.66)	PS/PC (0.66)
	Worst	TOT/C (0.00)	TOT/C (0.56)	ITER/PC (0.34)	TOT/C (0.20)	TOT/PC (0.51)	ITER/PC (0.49)
	Baseline	SIMP/C (0.19)	SIMP/C (0.76)	SIMP/C (0.72)	SIMP/C (0.58)	SIMP/C (0.62)	SIMP/C (0.64)

Column C represents the corresponding bug class, where each number corresponds to a specific type of issue. The classes are as follows: (1) for configuration issues, (2) for database-related issues, (3) for functional issues, (4) for GUI-related issues, (5) for network issues, (6) for performance issues, (7) for permission/deprecation issues, (8) for security issues, and (9) for test code-related issues. Bold highlights the best-performing configuration per class.

performance, indicating that under certain conditions, the collaborative nature of these prompts can enhance model effectiveness. This mixed result highlights that while these prompts may pose challenges, they also hold potential for improved accuracy in specific scenarios where their complexity aligns well with model capabilities and task requirements. Thus, selecting the appropriate prompt type remains essential, as it can either impede or elevate performance depending on the context.

Finding 11: Prompt engineering strategies, which incorporate both prompt types and output formatting instructions, significantly influence model performance in classifying bug types, improving the baseline 53 over 54 cases, though their effectiveness varies across different categories and models. This finding underscores that while prompt design can enhance classification, the impact is not uniform—certain bug types benefit more from specific prompt types and output formats than others, and no single strategy proves universally optimal across all model-class combinations. This variability highlights the need for tailored prompt engineering approaches that leverage each model’s unique strengths, adapting to the specific classification challenges posed by different bug types.

4.3.3 Impact of Prompt Engineering Strategies on the Alignment of Model-Generated Outputs with Human Annotations.

Approach: To evaluate the impact of prompt engineering strategies on the alignment of model-generated outputs with human annotations, we measure the agreement between model outputs and

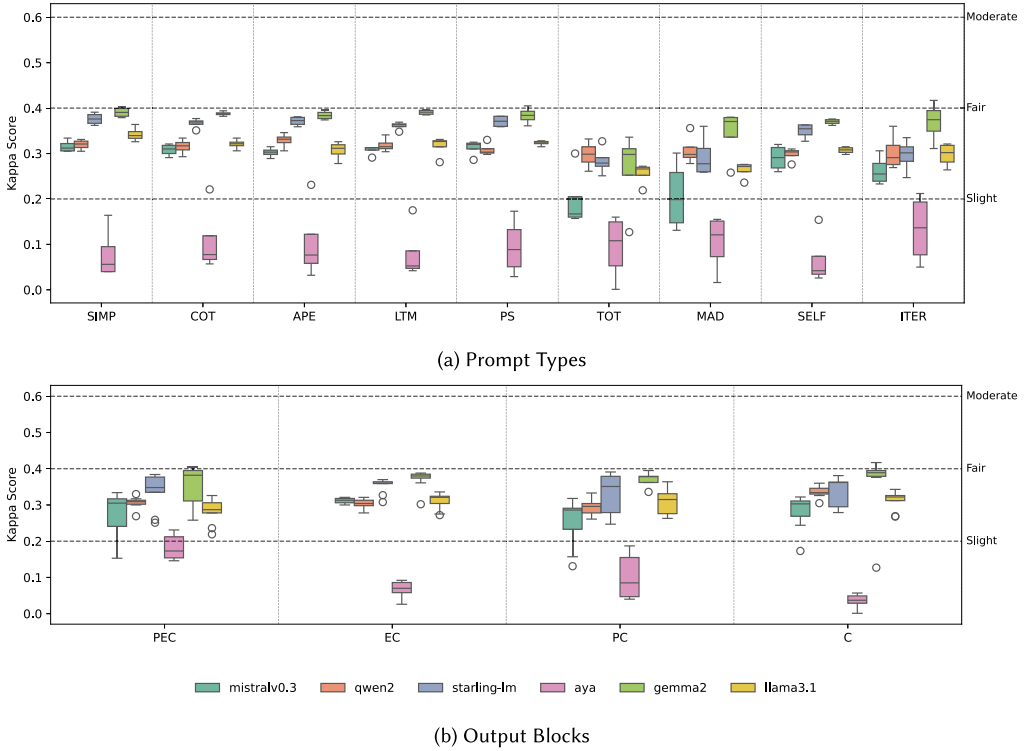


Fig. 6. Distribution of agreement to human annotations with relevant **Cohen's Kappa** agreement levels.

human annotations from the gold-standard dataset using **Cohen's Kappa**. This metric quantifies the consistency of LLM-generated bug report categories with human judgments, following practices in qualitative research [23] and LLM-assisted content analysis [16]. We conduct a systematic evaluation across various prompt types and output block configurations, calculating **Cohen's Kappa** for each model-configuration pair to assess alignment with human annotations. We apply ANOVA test to assess whether prompt engineering strategies significantly affect Kappa values. A p-value below 0.05 indicates that certain strategies improve alignment with human annotations and classification performance, while non-significant results ($p > 0.05$) suggest no substantial effect. This analysis helps identify the most effective configurations. We report Cohen's Kappa values and their interpretations (e.g., poor to almost perfect agreement) as outlined in previous studies [80], providing insights into how each model and prompt strategy aligns with human judgment in bug classification.

Results: Figure 6 presents the Cohen's Kappa results for different prompt types and output block configurations. The analysis reveals that the impact of prompt engineering strategies and output block configurations on model performance varies across different models. For models like Gemma2, Llama3.1, Starling-LM, and Mistral, incorporating different prompt types improves performance. Specifically, Gemma2 achieves its highest Kappa score of 0.42 with the ITER prompt type, while Starling-LM consistently records Kappa scores above 0.35 across most prompt types, with some reaching 0.40 (for COT). The ANOVA results support these findings, with Gemma2 ($p = 0.0033$), Llama3.1 ($p = 6.71e-06$), Starling-LM ($p = 3.92e-06$), and Mistral ($p = 0.000377$) all showing

significant differences in Kappa values across prompt types, indicating that prompt engineering strategies enhance classification performance for these models. In contrast, Aya and Qwen2 show little to no improvement with prompt engineering through prompt types, as reflected by non-significant p-values (Aya: $p = 0.9522$, Qwen2: $p = 0.6184$). These results suggest that for these models, adjustments to the prompt type do not significantly affect classification outcomes.

The analysis of output block configurations also shows mixed results. For models like Mistral ($p = 0.1431$), Llama3.1 ($p = 0.1487$), Gemma2 ($p = 0.9215$), and Starling-LM ($p = 0.7686$), output block configurations do not significantly impact performance, as indicated by p-values greater than 0.05. However, Qwen2 ($p = 0.00027$) and Aya ($p = 3.95e-09$) show significant improvements with specific configurations. For example, Qwen2 achieves Kappa scores ranging from 0.26 to 0.33 in the PC block, and Aya performs best in the PEC block with Kappa scores between 0.146 and 0.231. These findings suggest that, for certain models, specific output block configurations can notably improve alignment with human annotations.

Overall, the interpretation of Kappa values reveals that most prompt engineering strategies result in “Slight” to “Fair” agreement with human annotations. In the best cases, some configurations achieve “Moderate” agreement, while the worst cases indicate “Poor” agreement. This underscores the variability in model performance and indicates that while prompt engineering strategies can enhance model alignment with human judgments, their effectiveness is not universal. The impact varies depending on the model, suggesting the need for tailored approaches to optimize classification accuracy and alignment with human judgments.

Finding 12: Prompt engineering strategies improve model performance in aligning model-generated labels with human annotations but show variability across different models. Models like Gemma2, Llama3.1, Starling-LM, and Mistral show significant gains with specific prompt types (e.g., ITER, COT), while Aya and Qwen2 see minimal impact. Output block configurations also enhance performance for Qwen2 and Aya but not for other models. Overall, the effectiveness of prompt engineering varies, highlighting the need for tailored approaches based on model characteristics to improve alignment with human annotations.

RQ3-Summary: Prompt engineering strategies, including both prompt types and output formatting instructions, have a significant impact on the performance of various LLMs in classifying bug reports. Variability in model outputs affects precision, recall, and F1-scores, with some tailored prompts leading to more consistent and effective results, thereby demonstrating superior performance. Integrating lightweight post-processing techniques enhances classification consistency, particularly in F1-score and recall, while maintaining stable precision levels. Notably, no single model excels across all bug categories, highlighting the potential benefits of an ensemble approach. Overall, these insights underscore the critical role of thoughtful prompt design in improving the alignment of LLM outputs with human evaluations.

4.4 RQ4: To What Extent Can Model-Generated Bug Categorizations Serve as a Validation Approach for Human-Labeled Bug Categories?

4.4.1 Inter-Model Agreement.

Approach: To evaluate model consistency in classifying bug report categories, we use Fleiss’ Kappa to measure inter-model agreement across all configurations in our study. This analysis enables us to determine whether different configurations result in consistent or inconsistent categorizations across models. We compute Fleiss’ Kappa for each prompting strategy (prompt type and output block configurations) by constructing a contingency table that counts how often each configuration

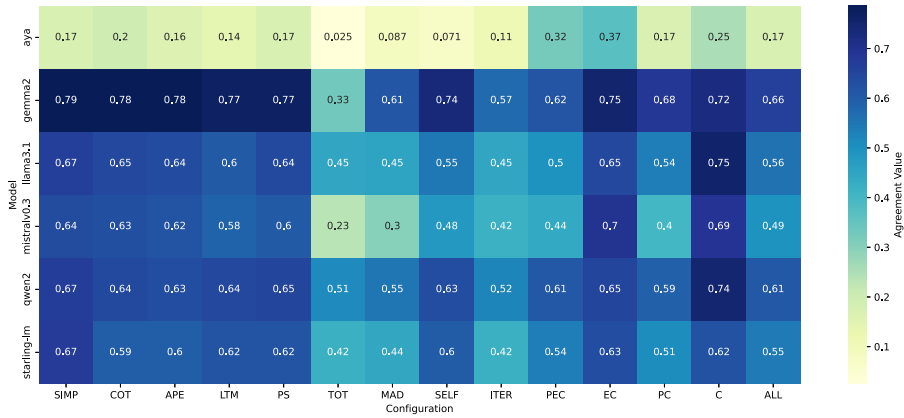


Fig. 7. Rater agreements across different models under different prompt type and output block configurations.

assigns specific labels to the bug reports, allowing us to assess the consistency of model-generated classifications.

Results: Figure 7 displays the Fleiss' Kappa values for all configurations considered in our study, illustrating the level of inter-model agreement in classifying bug report categories. The configuration "ALL" evaluates model agreement across 36 setups (9 prompt types over 4 output block configurations), providing a comprehensive assessment of how consistently the models label bug reports under varying conditions. In contrast, the PEC, PC, EC, and C configurations focus specifically on varying output block configurations, while the COT, ITER, LTM, and other configurations assess different prompt types for the given model.

The results show that the Gemma2 and Qwen2 models exhibit substantial agreement, with Fleiss' Kappa values of 0.61 and 0.66, respectively, across all 36 setups. This indicates a high level of consistency in their bug report labeling across various configurations. In contrast, the Starling-LM, Llama3.1, and Mistralv0.3 models demonstrate only moderate agreement, with Fleiss' Kappa values ranging between 0.49 and 0.56. This indicates that, while there is some consistency in how these models label bug reports, there is also a notable degree of variability. The Aya model is the exception, with a Fleiss' Kappa value of just 0.17, indicating only slight agreement. This low level of consistency suggests that the Aya model struggles to provide reliable categorizations, raising concerns about its effectiveness in accurately labeling bug reports.

The results also reveal that the type of prompt used significantly impacts model agreement and consistency. Prompt types that encourage step-by-step reasoning, such as COT, APE, LTM, and PS, lead to substantial agreement across models, suggesting that detailed reasoning enhances inter-model alignment. Similarly, the SIMP prompt type, which directly instructs models to categorize, shows strong agreement across most models, indicating that clear and direct instructions foster consistent performance. In contrast, the SELF prompt type, which incorporates multiple reasoning paths, exhibits moderate agreement, suggesting that while it can still produce reliable results, the added complexity introduces some variability, reducing consistency. Collaborative decision-making prompts, such as TOT, MAD, and ITER, tend to produce greater variability, with agreement levels ranging from fair to moderate. While these prompts can add depth to the responses, they also introduce inconsistencies among models.

Output block configurations like C and EC exhibit the highest agreement rates across multiple models, likely because the inclusion of both a category and an explanation provides clearer context,

enhancing consistency, which promotes consistency. In contrast, the PC and PEC configurations generally show lower agreement rates, suggesting that the inclusion of a process block complicates alignment among models.

Finding 13: The analysis indicates that certain models excel in consistent bug report labeling, with prompt types and output block configurations significantly influencing inter-model agreement rating. Prompt types, such as step-by-step reasoning types (e.g., COT, APE), enhance alignment by providing clear, detailed guidance, leading to more consistent outputs. Similarly, output block configurations like C and EC, which focus on both category generation and explanation, promote consistency across models. In contrast, collaborative decision-making prompt types, such as those using TOT, MAD, and ITER prompts, introduce variability by adding depth to the responses but also creating inconsistencies among models. By analyzing and comparing how models categorize bug reports, it's possible to identify areas where they agree (alignment) or disagree (discrepancy). These comparisons can help uncover potential issues in the human-assigned labels, such as errors or inconsistencies. The insights gained can then be used to improve the labeling process.

4.4.2 Evaluating Model Agreement and Discrepancies.

Approach: Our analysis focuses on examining the agreements and disagreements among model-generated bug category labels to identify inconsistencies and consistencies. Inconsistencies may indicate misclassifications or ambiguous bugs, while consistencies may reveal patterns where specific features are reliably associated with certain categories. The objective of this process is to identify inaccuracies or misclassifications in the human-generated labels, helping to identify potentially mislabeled dataset entries. By leveraging model outputs in conjunction with human labels, we aim to validate and correct dataset labels, ultimately ensuring that the dataset used for bug categorization is accurate and consistent.

To detect inconsistencies and consistencies, we use a majority voting method with thresholds to capture variations in model outputs. Disagreements are flagged when the proportion of models generating different labels exceeds a threshold, indicating potential inconsistencies that require review. Conversely, when the models' agreement exceeds a threshold their outputs are combined into a unified label, which is then used to validate human annotations and serve as the recommended categorization for the bug report.

Previous findings in our study underscore the importance of selecting appropriate prompt types and output block configurations to enhance model performance and improve the reliability of bug report labeling. Building on these insights, we adopt a systematic approach to identify the prompt types and output block configurations to be used in the voting process, which are as follows:

- *Select All Possible Generations:* We begin by including all outputs generated across different configurations.
- *Filter by Output Block Configuration:* Next, we refine the selection to focus on outputs that adhere to specific output block configurations.
- *Filter by Prompt Type:* Following that, we narrow down the population further by selecting outputs that utilize the same prompt type.
- *Filter by Model:* Finally, we select outputs generated by the same model to ensure that the voting process considers outputs from similar sources.

Within each configuration, we apply thresholds ranging from 50% to 100%, assessing both agreement and disagreement levels. These thresholds represent the minimum percentage of model agreement or disagreement required to flag a bug report for review. This approach, which applies majority voting with thresholds across different configurations, ensures that each bug report is evaluated in multiple contexts, enhancing coverage and consistency. This allows us to identify and

Configuration	SIMP	376	298	261	222	191	166	110	79	50	27	11
	COT	373	299	271	238	201	174	120	89	59	37	13
	APE	383	302	265	233	197	172	117	90	61	35	10
	LTM	357	285	249	223	196	171	108	72	36	17	8
	PS	374	300	266	227	195	162	110	86	50	23	11
	TOT	313	220	190	162	138	107	51	33	16	4	0
	MAD	348	266	229	190	159	135	90	66	34	19	3
	SELF	367	285	245	210	179	152	101	66	37	14	3
	ITER	324	250	218	183	159	133	76	57	30	19	5
	ALL	335	281	236	190	162	131	100	63	31	7	0
	PEC	366	313	250	206	183	146	122	95	56	24	4
	EC	360	311	252	204	170	132	101	80	51	29	6
	PC	333	280	227	175	152	131	93	67	37	9	3
	C	332	287	243	203	171	136	90	41	18	2	0
	mistralv0.3	463	420	367	324	281	264	217	155	101	48	17
	qwen2	490	451	409	371	327	306	271	240	193	146	104
	starling-lm	398	356	325	282	241	219	175	139	101	49	25
	aya	649	501	360	252	172	140	81	43	18	5	2
	gemma2	445	409	376	355	319	302	274	231	175	84	34
	llama3.1	473	428	383	339	301	288	255	206	162	89	47
	Combined	845	776	719	653	592	564	497	421	328	205	141
		50	55	60	65	70	75	80	85	90	95	100
		Threshold										

Fig. 8. Agreement analysis: Number of unique bug reports highlighted for further investigation under selected configuration and threshold.

prioritize bug reports that require closer examination, directing manual review efforts toward cases with significant discrepancies or notable agreements among the models.

Following the voting process approach, we perform a targeted manual review of the bug reports identified at selected thresholds. This review involves examining a subset of reports, chosen based on experimental thresholds, to evaluate the accuracy and reliability of model-generated labels by comparing them to human-assigned categories. By addressing the discrepancies and agreements identified during the review, we aim to guide the development of improved labeling methodologies and establish a reliable framework for evaluating human-applied categories. Thus, this comprehensive process, combining automated analysis of model (dis)agreement with targeted manual review, provides a concrete methodology for addressing the inherent challenge of potential inaccuracies in human-labeled datasets, directly tackling the limitation identified earlier in this work.

Results: Agreements. Figure 8 illustrates the distribution of unique bug reports identified for further investigation based on the chosen configurations and thresholds. Each cell represents the number of bug reports flagged by the configuration, reflecting consensus among the models on their categorization. To establish a useful cutoff, we focus on bug reports associated with higher thresholds, as these indicate stronger agreement and potentially more reliable categorization. We specifically use the “Combined” row to determine this cutoff, as it reflects the total number of unique bug reports across all configurations for each threshold. At the 100% threshold, the “Combined” row identifies 141 unique bug reports. Given the total of **1,024 bug reports**, this selection appears to represent a reasonable sample for further analysis.

The manual analysis of the 141 bug reports reveals several significant findings. Among these reports, the model-generated label is considered more suitable in 88 cases compared to the labels assigned by human annotators. A prominent theme we identify is the misclassification of configuration issues. In 61 instances, human annotators incorrectly categorize these reports, despite clear indications in the bug descriptions relating to build issues, configuration files, and problems with external libraries or incorrect file paths. Other misclassifications include eight reports that should have been labeled as functional issues, four as performance issues, three as database-related issues, two as security issues, and one as test-related issues.

Conversely, in 43 reports, the labels assigned by human annotators are found to be correct. Of these, 32 are misclassified by the models as functional issues. Notably, 31 reports are inaccurately

labeled as configuration issues, with one labeled as a database issue. Upon closer examination of these 31 mislabeled cases, it becomes evident that all mention terms related to files, folders, XML, or other property files. While the prompt used to define configuration issues explicitly refers to problems such as 'bugs related to build configuration files, issues with external libraries that need updating or fixing, and incorrect directory or file paths in XML or manifest artifacts,' the issues we observed in the misclassified bug reports are actually functional problems, not configuration errors. These issues, occurring within files, folders, XML, or manifest artifacts, involve the software's behavior or functionality rather than the configuration of the system. For example, the malfunction of the XERCESJ parser,³ which fails to parse an XML file, is mistakenly categorized by the models as a configuration issue, whereas it is fundamentally a functional issue. Similarly, another bug report⁴ describes an issue in Eclipse that generates an incorrect structure and extra folders; the models again misclassify this as a configuration bug, despite it being a functional malfunction. This discrepancy between the prompt's definition of 'configuration issues' and the actual nature of the bugs may explain why the models misclassify these reports, confusing functional issues for configuration problems. Additionally, the model misclassifies six GUI-related issues, three permission/deprecation issues, one security issue, and one network issue that human annotators labeled correctly.

We also observed that, for five reports, both the model-generated labels and the human labels appear suitable. This suggests that multi-labeling may be necessary for certain cases, where bug reports span multiple categories. However, for three reports, neither of the labels seem accurate; two of these should instead be categorized as permission issues and one as a functional issue. Additionally, one report lacks sufficient information to determine an appropriate label, and another report is actually a refactoring request rather than a bug. The full list of these cases can be found in online supplementary material (<https://zenodo.org/records/14195747>), which details the determined label for each bug report along with the authors' comments.

Finding 14: Leveraging the agreements of models serves as a valuable validation approach for human-labeled categories, as evidenced by the analysis showing that in 88 of 141 cases, model-generated labels are more suitable than those assigned by human annotators. This systematic evaluation suggests that model predictions can complement human annotations by providing an additional layer of validation. Leveraging model-based insights alongside human annotations can therefore enhance the accuracy, consistency, and reliability of the labeled dataset, leading to improved bug categorization and potentially informing better guidelines for human annotators.

Results: Disagreements. Figure 9 illustrates the distribution of unique bug reports categorized by the level of disagreement among model-generated labels, highlighting the instances where models diverge in their classifications and emphasizing the need for further investigation into these discrepancies. To establish a useful cutoff, we concentrate on bug reports linked to higher thresholds, which signify stronger disagreement among model-generated labels. We utilize the "Combined" row for this determination, as it represents the total number of unique bug reports across all configurations at each threshold. At the 75% threshold, the "Combined" row identifies 34 unique bug reports.

The manual analysis of these 36 bug reports reveals that 12 of these cases have accurate human annotations, while the remainder is considered misclassified. A significant portion of these reports contain limited information, making it challenging to determine an accurate category. To gain a better understanding of the nature of these bugs, we review the full discussion history from the repository, which led to the identification of more appropriate category labels for 20 of these reports.

³<https://issues.apache.org/jira/browse/XERCESJ-42>.

⁴https://bugs.eclipse.org/bugs/show_bug.cgi?id=36465.

Configuration	SIMP	268	135	81	42	16	2
	COT	264	134	85	48	15	3
	APE	274	140	92	46	17	6
	LTM	285	161	101	54	31	8
	PS	279	143	81	42	16	4
	TOT	422	247	163	90	42	9
	MAD	364	190	110	52	25	5
	SELF	304	165	104	53	20	5
	ITER	341	154	78	39	17	4
	ALL	279	179	104	51	14	1
	PEC	246	147	83	25	5	0
	EC	280	174	98	37	11	1
	PC	306	194	110	49	16	1
	C	309	206	111	39	12	3
	mistralv0.3	169	95	44	14	2	1
	qwen2	128	73	29	6	1	1
	starling-lm	152	81	45	14	4	1
	aya	347	181	86	30	4	0
	gemma2	79	27	7	3	0	0
	llama3.1	154	83	44	13	2	1
	Combined	745	575	411	233	117	36
		50	55	60	65	70	75
		Threshold					

Fig. 9. Disagreement analysis: Number of unique bug reports highlighted for further investigation under selected configuration and threshold.

From the manual analysis of the disagreements, we observed that the identified bug reports are indeed more contentious and debatable, especially when compared to the previous agreement-based analysis, which resulted in a smoother manual labeling process with fewer unclear cases. Therefore, we can confirm that the disagreement-based approach effectively highlights more complex and ambiguous cases.

Finding 15: Leveraging the disagreements of models can be especially useful for identifying contentious or poorly detailed reports. This approach highlights the value of model-generated labels in flagging ambiguous cases that may require further examination, ultimately improving the quality of bug categorization and ensuring more accurate labeling in challenging scenarios.

RQ4-Summary: This RQ investigates how model-generated categorizations can serve as a systematic approach to validate the correctness of human-applied labels. We first examined factors influencing inter-model agreement (Section 4.4.1), finding that elements like basic prompting and step-by-step reasoning enhance model consistency, providing more reliable outputs for validation. Subsequently, (Section 4.4.2), we analyzed patterns of agreement and disagreement among the model-generated labels themselves. This process acts as a validation mechanism: comparing them against the human-assigned labels allows us to identify potential errors in the human annotations (flagged by model disagreements or strong model consensus differing from the human label) and ultimately help to assess and improve the correctness of the original dataset labels. Our findings demonstrate this combined approach effectively identifies potential misclassifications and enhances dataset quality.

5 Threats to Validity

One potential threat to internal validity concerns data leakage in the LLMs used in our study. Specifically, it is possible that these models may have prior knowledge of the bug reports in the dataset. However, we do not consider this to be a decisive threat to internal validity, as the bug types associated with these reports are not explicitly specified within the reports themselves. Instead, the bug categorization is inferred manually based on the taxonomy developed by Catolino et al. [13].

This taxonomy is applied independently of any predefined or explicitly stated categories within the bug reports, as the reports do not include dedicated fields for categorization.

The prompt template used in our experiments may not represent the optimal choice for each model. Additionally, the wording of prompts has been shown to significantly influence the performance of LLMs. To mitigate this threat, we carefully design our prompts based on existing literature and integrate specific instructions for each prompt type (Table 3), as recommended in the corresponding studies. Furthermore, we execute experiments with six different models, each with various configurations of the prompt templates, and recorded all responses to assess the sensitivity of model performance to different prompt formulations. To further mitigate threats to validity and promote reproducibility, we have made both our scripts and all data publicly available, allowing others to replicate and verify our findings.

Controlling LLM output through a combination of prompt engineering and lightweight post-processing may not be applicable universally to all models. However, the methodology we used to analyze adherence to instructions, consistency with expected category labels, and evaluation of common output formats can be generalized and easily adapted to other models. Choosing this lightweight regex approach, however, means our study does not compare its performance against more sophisticated post-processing methods (e.g., training secondary classifiers or using embedding similarity), which constitutes a limitation inherent to our specific focus on prompt engineering impacts.

An internal threat to validity may rise from the manual review of bug report categories for RQ4. The risk here is that human error or subjective judgment could impact the categorization process, leading to misclassifications. To mitigate this threat, we consider labels that are consistently generated by the models, cross-reference them with previous annotations, carefully examine the bug reports, and, when necessary, review the full discussion history from the repositories to gain a deeper understanding of the nature of these bugs. The complete list of cases, including the determined label for each bug report along with the authors' comments, has been made available for further validation in online supplementary material (<https://zenodo.org/records/14195747>).

A potential threat to internal validity arises from our selection of only open-source LLMs with 7B-9B parameters from Ollama. This limits model diversity, as performance can vary significantly across different model sizes, platforms, and training data. By excluding other models, particularly those from commercial providers or with different parameter sizes, we may not capture the full range of LLM behaviors. Hence, our results may not fully generalize to other model sizes or platforms, and future work could extend this analysis to a broader selection of models.

The main external threat is the use of a single dataset to evaluate the LLM-based bug report categorization, which may limit the applicability of our approach. To mitigate this threat, we select a benchmark covering a wide range of real-world bug types, collected from 119 software projects across three large and varied ecosystems: Mozilla, Apache, and Eclipse.

6 Discussions

Based on our findings from the study, we summarize a series of implications, which could suggest better practice of leveraging LLMs in bug report categorization.

❶ *Need for Post-Processing and Tailored Approaches*: Significant variation in adherence to output instructions across models necessitates post-processing and the exploration of different prompting strategies. Tailored approaches are essential for accurately extracting bug categories from model outputs, especially when dealing with models that show inconsistent adherence to output instructions.

② *Extended Regex for Non-Standard Formats*: Simple regex approaches fail when models provide answers in non-standard formats. This indicates the need for enhanced and flexible regex rules to address the variability observed across different prompt types and improve bug category extraction accuracy. While our extended regex effectively addresses common formatting inconsistencies observed in this prompt engineering-focused study, comparing its performance against more advanced machine learning or semantic-based post-processing techniques remains a valuable direction for future research.

③ *Prompt Type Matters*: Adherence to formatting instructions alone does not guarantee accurate category prediction. The choice of prompt type plays a crucial role in improving model performance and ensuring that outputs align with expected categories. A more refined approach is needed to address model-specific nuances.

④ *Improved Consistency for Extraction*: While strict formatting adherence remains model-dependent, prompt engineering with structured output instructions (“P” or “E”) demonstrably induced sufficient output consistency to make the category label extractable.

⑤ *Impact of Prompt Engineering*: Prompt engineering strategies, especially those that incorporate additional output formatting instructions, greatly enhance model performance in categorizing bug reports. While improvements are significant, their impact varies across different models and bug types, requiring model-specific adjustments to prompt strategies.

⑥ *Variable Effectiveness of Prompt Engineering*: Prompt engineering can significantly improve classification performance, but its impact varies across different models and bug categories. A more tailored approach to prompt engineering is needed, rather than a one-size-fits-all strategy, adapting to each model’s strengths and the specific challenges of different bug types.

⑦ *Optimizing Bug Categorization for Challenging Categories*: Even with peak performances, models struggle with certain bug types, such as network or functional issues. To enhance model performance, strategies such as fine-tuning models, experimenting with larger models, and optimizing context size should be considered. Adopting a multi-label approach allows bug reports belonging to multiple categories to be labeled appropriately [90], reflecting their complexity. Expanding the taxonomy to include additional labels or creating subtypes for broader categories, such as functional issues, can also help capture finer distinctions. Incorporating external knowledge, such as domain-specific ontologies or historical bug data, and leveraging advanced prompt engineering techniques to improve context understanding are further potential avenues for improvement.

⑧ *Leveraging Model-Generated Labels for Validation*: Leveraging model-generated labels in combination with human annotations provides valuable validation for bug categorization. By analyzing areas of agreement, we can enhance the reliability of bug report categorization, ultimately improving human labeling processes.

⑨ *Disagreement Analysis for Ambiguous Cases*: Leveraging model disagreements proves particularly useful in identifying ambiguous or contentious bug reports. Focusing on these cases helps prioritize further investigation, improving categorization quality, and ensuring accuracy in challenging scenarios.

7 Related Work

7.1 Automatic Classification of Bug Reports

While many approaches aim to classify issues, most primarily focus on binary classification—such as determining whether an issue report is a bug or a non-bug [6, 61, 63, 97]. This binary classification, while useful, limits the depth of analysis. It fails to provide insights into the nature of submitted bugs, treating all bugs uniformly without considering their specific types. Understanding the distinct categories of bugs can significantly enhance the identification of the most suitable developer for

debugging tasks, thereby accelerating the bug analysis and resolution process [13]. To address the limitations of binary classification, several studies have expanded efforts to categorize bugs into various types and labels, applying defect classification schemas such as mandelbugs and BOH [32, 33], **Orthogonal Defect Classification (ODC)** [18], and Seaman’s taxonomy [67] to support more detailed automated bug categorization. Building on these schemas, Thung et al. [76] and Huang et al. [41] classified defects using the ODC framework, while Xia et al. [89] and Du et al. [27] applied classifications based on mandelbugs and BOH. More recently, Koksall et al. [49] employed Seaman’s taxonomy to categorize bugs, further refining automated classification approaches.

In addition to defect classification schemas, other classification techniques categorize bug reports into specific types and categories, enhancing the granularity of categorization. Herzig et al. [37] classified issues into six different issue categories—bug, feature request, improvement request, documentation request, refactoring request, and others. Tan et al. [74] identified three primary bug types—semantic, security, and concurrency issues—that underlie most significant software bugs. Chawla and Singh [14] proposed a system to identify defect categories, such as security and crash-related issues, within bug reports. Kochhar et al. [47] employed classifiers to automatically check if an issue needs to be reclassified and if so, into one of the 13 categories defined by Herzig et al. [37]. Zibran [99] categorized bug reports into 22 specific categories focusing on API usage factors such as concurrency and memory management. The CaPbug model [1] predicted categories such as program anomaly, GUI, performance, and test-code issues [1]. Kallis et al. [46] introduced “Ticket Tagger” to classify issues as bug reports, enhancements, or questions based on titles and descriptions [46]. Similarly, Catolino et al. [13] developed a classification schema encompassing nine categories, including configuration, network, database, GUI, performance, permission/deprecation, security, program anomaly, and test-code issues [13], which is also utilized in our work.

With the rise of transformers and LLMs, recent studies have increasingly explored the use of PLMs for automatic issue labeling. Wang et al. [83] demonstrated that PLMs like BERT outperform traditional models, such as Bi-LSTM, CNN, and RCNN, for multi-label issue classification. Nadeem et al. [57] fine-tuned RoBERTa for classifying issues into bugs, enhancements, and questions, showing the power of PLMs for high-accuracy predictions. Expanding on this, Trautsch and Herbold [78] used the seBERT model to predict issue types. Da Silva et al. [24] applied DistilBERT to classify exception-handling bugs, while Meng and Visser [55] used BERT for bug validity classification. Yokoyama et al. [94] focused on security bug identification, evaluating multiple techniques, including BERT, N-gram IDF, and ChatGPT. Additionally, Alhindi et al. [4] fine-tuned ALBERT to classify issues as bugs, improvements, or features within a Jira plugin. Colavito et al. [20] investigated lightweight fine-tuning techniques, such as few-shot learning, and showed the effectiveness of GPT-like models for automating issue labeling without the need for fine-tuning [22]. Ebrahim and Joy [28] proposed a RoBERTa-based classifier with adapters, adding extra layers to improve classification performance. Li et al. [51] evaluated GPT-3.5 for categorizing bug reports using prompt engineering (CoT and ToT) and fine-tuning, classifying reports into BOH, ARB, NAM, and non-bug. Du et al. [26] employed OpenAI’s textembedding-ada-002 to represent bug reports as vectors, which were then classified into the same categories using a Feed-Forward Neural Network. Colavito et al. [21] found that RoBERTa-based classifiers achieve state-of-the-art performance in issue classification and explored the impact of data quality on classifier performance using generic data quality filters. Aracena et al. [7] fine-tuned the ChatGPT-3 model to classify issues as bugs, features, or questions, while Rejithkumar et al. [65] used a text-to-text generation approach with the T5 model for issue report classification.

Our study differs from these works in several ways: (i) we focus on fine-grained bug report categorization rather than general categories; (ii) we explore the use of open-source LLMs, ranging from 7B to 9B parameters, without fine-tuning; (iii) we examine a broader range of prompt engineering

strategies, many of which have not been explored for bug report categorization; (iv) we explore dataset label correctness by leveraging model-generated labels to validate human annotations.

7.2 Data Annotation

Understanding and addressing label uncertainty is central to improving data annotation processes. Hsueh et al. [40] showed that annotators, despite good intentions, can introduce errors or inconsistencies due to misunderstandings or changing interpretations over time. Sheng et al. [69] emphasized the importance of acquiring multiple labels for data items, especially when labeling is imperfect, and showed that selective acquisition of additional labels enhances annotation quality by mitigating the effects of ambiguities and subjective interpretations. Aroyo et al. [8] expanded on this idea, showing that disagreements among annotators can signal ambiguity in the data or variability in human perception. Their CrowdTruth approach harnesses disagreement to pinpoint problematic areas in annotation tasks, improving overall quality by embracing the subjectivity inherent in human interpretation. Inspired by these seminal studies on annotator behavior, our research proposes leveraging LLMs as annotators. By analyzing their agreements and disagreements among LLM-generated labels, we aim to enhance the validation of bug report categories annotated by humans. This method helps identify unclear reports and detect potential misclassifications, thereby improving the overall annotation process.

Building on these insights into annotator behavior, machine learning techniques have emerged to bridge the gap between data labeling and model performance. Gordon et al. [30] introduced jury learning, a supervised approach that explicitly incorporates annotator disagreement into model training. By modeling each annotator as a juror, this method allows practitioners to specify whose perspectives a classifier should reflect and in what proportions. This ensures that models can account for the diversity of human viewpoints and address contested or subjective tasks more effectively.

In parallel, LLMs are increasingly utilized to supplement or replace traditional annotation workflows. Chiang and Lee [17] proposed using LLMs to evaluate text quality, offering a scalable alternative to human evaluation while minimizing human exposure to objectionable content. Similarly, He et al. [35] demonstrated that LLMs, such as GPT-3.5, can be tailored for specific annotation tasks by providing detailed instructions, enabling them to perform with precision comparable to human annotators. While models like GPT-4 and Llama 3.1 show promise in aligning with human annotators, Movva et al. [56] cautioned that the process of understanding and categorizing nuanced safety concerns in chatbot interactions remains inherently subjective. They proposed leveraging the diversity of opinions within the annotator pool rather than enforcing unanimous agreement, which could lead to more robust safety evaluations and improved model alignment over time. By addressing label uncertainty and integrating advanced methods such as jury learning and LLMs, researchers are redefining the annotation process. Our study contributes in this direction by exploring how LLM-generated categorization of bug reports compares to human annotations, offering insights into the alignment between automated and manual categorization processes.

8 Conclusions

This study explores the potential of fine-grained bug report categorization using LLMs combined with prompt engineering techniques. Our findings reveal that while LLMs show promise in categorizing bug reports, their performance varies significantly depending on the prompt types, output formatting instructions, and model-specific characteristics.

Our investigation also emphasizes the importance of tailored prompt engineering strategies. Specific prompt types and configurations, such as CoT reasoning and output blocks, significantly

enhance model performance in aligning with predefined categories and adhering to output instructions. Nevertheless, the variability in model behavior necessitates iterative experimentation to optimize classification performance for different bug types.

By integrating prompt engineering with lightweight post-processing techniques, we achieved notable improvements in output consistency and alignment with human annotations. Additionally, our analysis revealed that prompt engineering, including structured instructions for additional output blocks (e.g., Process and Explanation), can impact models' adherence to formatting instructions and the reliability of bug categorization, though often in model-dependent ways. However, challenges remain, particularly in handling inconsistent outputs and addressing specific bug categories like network or functional issues, where model performance is less robust.

We observed that leveraging model agreements provides a valuable validation mechanism for human-labeled categories, with models outperforming human annotations in a substantial number of cases. Additionally, analyzing disagreements between models highlights contentious or poorly detailed reports, helping prioritize cases for further manual review. These insights underscore the complementary roles of model-generated and human annotations in improving bug categorization.

In summary, this empirical study demonstrates the potential of LLMs and prompt engineering for fine-grained bug report categorization, providing a pathway for more accurate, efficient, and scalable categorization processes. Future work could explore model fine-tuning, enhanced post-processing techniques, and further refinements in prompt engineering to address persistent challenges and expand the applicability of LLMs in software engineering contexts.

Acknowledgment

The authors thank TUBITAK for their support.

References

- [1] Hafiza Anisa Ahmed, Narmeen Zakaria Bawany, and Jawwad Ahmed Shamsi. 2021. Capbug-a framework for automatic bug categorization and prioritization using nlp and machine learning algorithms. *IEEE Access* 9 (2021), 50496–50512.
- [2] Toufique Ahmed and Premkumar Devanbu. 2023. Better patching using LLM prompting, via self-consistency. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1742–1746. DOI: <https://doi.org/10.1109/ASE56229.2023.00065>
- [3] Khubaib Amjad Alam, Ashish Jumani, Harris Aamir, and Muhammad Uzair. 2024. ClassifAI: Automating issue reports classification using pre-trained BERT (bidirectional encoder representations from transformers) models. In *Proceedings of the 3rd ACM/IEEE International Workshop on NL-Based Software Engineering*, 49–52.
- [4] Waleed Alhindi, Abdulrahman Aleid, Ilyes Jenhani, and Mohamed Wiem Mkaouer. 2023. Issue-labeler: An ALBERT-based Jira plugin for issue classification. In *Proceedings of the 2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 40–43. DOI: <https://doi.org/10.1109/MOBILSoft59058.2023.00012>
- [5] Xavier Amatriain. 2024. Prompt design and engineering: Introduction and advanced methods. arXiv:2401.14423. Retrieved from <https://arxiv.org/abs/2401.14423>
- [6] Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël Guéhéneuc. 2008. Is it a bug or an enhancement? A text-based approach to classify change requests. In *Proceedings of the 2008 Conference of the Center for Advanced Studies on Collaborative Research: Meeting of Minds*, 304–318.
- [7] Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A. Gerosa. 2024. Applying large language models API to issue classification problem. arXiv:2401.04637. Retrieved from <https://arxiv.org/abs/2401.04637>
- [8] Lora Aroyo and Chris Welty. 2014. The Three sides of CrowdTruth. *Human Computation* 1, 1 (Sep. 2014). DOI: <https://doi.org/10.15346/hc.v1i1.3>
- [9] Viraat Aryabumi, John Dang, Dwarak Talupuru, Saurabh Dash, David Cairuz, Hangyu Lin, Bharat Venkitesh, Madeline Smith, Kelly Marchisio, Sebastian Ruder, et al. 2024. Aya 23: Open weight releases to further multilingual progress. arXiv:2405.15032. Retrieved from <https://arxiv.org/abs/2405.15032>

- [10] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv:2204.05862. Retrieved from <https://arxiv.org/abs/2204.05862>
- [11] Shikhar Bharadwaj and Tushar Kadam. 2022. Github issue classification using BERT-style models. In *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering*, 40–43.
- [12] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Vol. 33, 1877–1901.
- [13] Gemma Catolino, Fabio Palomba, Andy Zaidman, and Filomena Ferrucci. 2019. Not all bugs are the same: Understanding, characterizing, and classifying bug types. 152 (2019), 165–181. DOI : <https://doi.org/10.1016/j.jss.2019.03.002>
- [14] Indu Chawla and Sandeep K. Singh. 2014. Automatic bug labeling using semantic information from LSI. In *Proceedings of the 2014 7th International Conference on Contemporary Computing (IC3)*. IEEE, 376–381.
- [15] An Ran Chen, Tse-Hsun (Peter) Chen, and Shaowei Wang. 2021. Demystifying the challenges and benefits of analyzing user-reported logs in bug reports. *Empirical Software Engineering* 26, 1 (Jan. 2021), 8. DOI : <https://doi.org/10.1007/s10664-020-09893-w>
- [16] Robert Chew, John Bollenbacher, Michael Wenger, Jessica Speer, and Annice Kim. 2023. LLM-assisted content analysis: Using large language models to support deductive coding. arXiv:2306.14924. Retrieved from <https://arxiv.org/abs/2306.14924>
- [17] Cheng-Han Chiang and Hung-yi Lee. 2023. Can large language models be an alternative to human evaluations? arXiv:2305.01937. DOI : <https://doi.org/10.48550/arXiv.2305.01937>
- [18] Ram Chillarege, Inderpal S. Bhandari, Jarir K. Chaar, Michael J. Halliday, Diane S. Moebus, Bonnie K. Ray, and Man-Yuen Wong. 1992. Orthogonal defect classification—a concept for in-process measurements. *IEEE Transactions on Software Engineering* 18, 11 (1992), 943–956.
- [19] Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. 2022. Issue report classification using pre-trained language models. In *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering*, 29–32.
- [20] Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. 2023. Few-shot learning for issue report classification. In *Proceedings of the 2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*. IEEE, 16–19. DOI : <https://doi.org/10.1109/NLBSE59153.2023.00011>
- [21] Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. 2024. Impact of data quality for automatic issue classification using pre-trained language models. *Journal of Systems and Software* 210 (Apr. 2024), 111838. DOI : <https://doi.org/10.1016/j.jss.2023.111838>
- [22] Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. 2024. Leveraging GPT-like LLMs to automate issue labeling. In *Proceedings of the 2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, 469–480.
- [23] Monica L. Coleman, Moira Ragan, and Tahani Dari. 2024. Intercoder reliability for use in qualitative research and evaluation. *Measurement and Evaluation in Counseling and Development* 57, 2 (2024), 136–146.
- [24] Antônio José A. da Silva, Renan G. Vieira, Diego P. P. Mesquita, João Paulo P. Gomes, and Lincoln S. Rocha. 2024. Towards automatic labeling of exception handling bugs: A case study of 10 years bug-fixing in Apache Hadoop. *Empirical Software Engineering* 29, 4 (Jul. 2024), 1–30. DOI : <https://doi.org/10.1007/s10664-024-10494-0>
- [25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805. Retrieved from <https://arxiv.org/abs/1810.04805>
- [26] Xiaoting Du, Zhihao Liu, Chenglong Li, Xiangyue Ma, Yingzhuo Li, and Xinyu Wang. 2024. LLM-BRC: A large language model-based bug report classification framework. *Software Quality Journal* 32, 3 (Sep. 2024), 985–1005. DOI : <https://doi.org/10.1007/s11219-024-09675-3>
- [27] Xiaoting Du, Zheng Zheng, Guanping Xiao, and Beibei Yin. 2017. The automatic classification of fault trigger based bug report. In *Proceedings of the 2017 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 259–265.
- [28] Fahad Ebrahim and Mike Joy. 2024. Few-shot issue report classification with adapters. In *Proceedings of the 3rd ACM/IEEE International Workshop on NL-Based Software Engineering (NLBSE '24)*. ACM, New York, NY, 41–44. DOI : <https://doi.org/10.1145/3643787.3648039>
- [29] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.
- [30] Mitchell L. Gordon, Michelle S. Lam, Joon Sung Park, Kayur Patel, Jeff Hancock, Tatsunori Hashimoto, and Michael S. Bernstein. 2022. Jury learning: Integrating dissenting voices into machine learning models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, New Orleans, LA, 1–19. DOI : <https://doi.org/10.1145/3491102.3502004>

- [31] Katerina Goseva-Popstojanova and Jacob Tyo. 2018. Identification of security related bug reports via text mining using supervised and unsupervised classification. In *Proceedings of the 2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 344–355.
- [32] Jim Gray. 1986. Why do computers stop and what can be done about it?. In *Symposium on Reliability in Distributed Software and Database Systems*, 3–12.
- [33] Michael Grottke and Kishor S. Trivedi. 2005. A classification of software faults. *Journal of Reliability Engineering Association of Japan* 27, 7 (2005), 425–438.
- [34] Muhammad Usman Hadi, Qasem Al-Tashi, Rizwan Qureshi, Abbas Shah, Amgad Muneer, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, et al. 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. *Authorea Preprints* 1 (2023), 1–26.
- [35] Xingwei He, Zhenghao Lin, Yeyun Gong, A.-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. arXiv:2303.16854. DOI: <https://doi.org/10.48550/arXiv.2303.16854>
- [36] Jueun Heo, Gibeom Kwon, Changwon Kwak, and Seonah Lee. 2024. A comparison of pretrained models for classifying issue reports. *IEEE Access* 12 (2024), 79568–79584.
- [37] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's not a bug, it's a feature: How misclassification impacts bug prediction. In *Proceedings of the 2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 392–401.
- [38] Thomas Hirsch and Birgit Hofer. 2022. Using textual bug reports to predict the fault category of software bugs. *Array* 15 (2022), 100189.
- [39] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* (2023).
- [40] Pei-Yun Hsueh, Prem Melville, and Vikas Sindhwani. 2009. Data quality from crowdsourcing: A study of annotation selection criteria. In *Proceedings of the NAACL HLT 2009 Workshop on Active Learning for Natural Language Processing*. Eric Ringger, Robbie Haertel, and Katrin Tomanek (Eds.). Association for Computational Linguistics, Boulder, Colorado, 27–35.
- [41] LiGuo Huang, Vincent Ng, Isaac Persing, Mingrui Chen, Zeheng Li, Ruili Geng, and Jeff Tian. 2015. AutoODC: Automated generation of orthogonal defect classifications. *Automated Software Engineering* 22 (2015), 3–46.
- [42] Dave Hulbert. 2023. *Using Tree-of-Thought Prompting to boost ChatGPT's Reasoning*. Zenodo. DOI: <https://doi.org/10.5281/zenodo.10323452>
- [43] Maliheh Izadi. 2022. Catiss: An intelligent tool for categorizing issues reports using transformers. In *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering*, 44–47.
- [44] Gaeul Jeong, Sunghun Kim, and Thomas Zimmermann. 2009. Improving bug triage with bug tossing graphs. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 111–120.
- [45] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. arXiv:2310.06825. Retrieved from <https://arxiv.org/abs/2310.06825>
- [46] Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. 2019. Ticket tagger: Machine learning driven issue classification. In *Proceedings of the 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 406–409.
- [47] Pavneet Singh Kochhar, Ferdian Thung, and David Lo. 2014. Automatic fine-grained issue report reclassification. In *Proceedings of the 2014 19th International Conference on Engineering of Complex Computer Systems*. IEEE, 126–135.
- [48] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Curran Associates Inc., New Orleans, LA, Article 1613, 15 pages.
- [49] Ömer Köksal and Bedir Tekinerdogan. 2022. Automated classification of unstructured bilingual software bug reports: An industrial case study research. *Applied Sciences* 12, 1 (Jan. 2022), 338. DOI: <https://doi.org/10.3390/app12010338>
- [50] Eric Lacker, Jongwook Kim, Akash Kumar, Lipika Chandrashekar, Srilaxmi Paramaiahgari, and Jasmine Howard. 2021. Statistical analysis of refactoring bug reports in Eclipse Bugzilla. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 9–13. DOI: <https://doi.org/10.1109/ASEW52652.2021.00015>
- [51] Chenglong Li, Zheng Zheng, Xiaoting Du, Xiangyue Ma, Zhengqi Wang, and Xinheng Li. 2024. KnowBug: Enhancing large language models with bug report knowledge for deep learning framework bug prediction. *Knowledge-Based Systems* 305 (Dec. 2024), 112588. DOI: <https://doi.org/10.1016/j.knosys.2024.112588>
- [52] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. 2023. Encouraging divergent thinking in large language models through multi-agent debate. arXiv:2305.19118. Retrieved from <https://arxiv.org/abs/2305.19118>

- [53] Jieyi Long. 2023. Large language model guided tree-of-thought. arXiv:2305.08291. DOI: <https://doi.org/10.48550/arXiv.2305.08291>
- [54] Marcos Macedo, Yuan Tian, Filipe Cogo, and Bram Adams. 2024. Exploring the impact of the output format on the evaluation of large language models for code translation. In *Proceedings of the 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering (FORGE '24)*. ACM, New York, NY, 57–68. DOI: <https://doi.org/10.1145/3650105.3652301>
- [55] Qianru Meng and Joost Visser. 2024. Which bug reports are valid and why? Using the BERT transformer to classify bug reports and explain their validity. In *Proceedings of the 4th European Symposium on Software Engineering (ESSE '23)*. ACM, New York, NY, 52–60. DOI: <https://doi.org/10.1145/3651640.3651648>
- [56] Rajiv Movva, Pang Wei Koh, and Emma Pierson. 2024. Annotation alignment: Comparing LLM and human annotations of conversational safety. arXiv:2406.06369. Retrieved from <https://arxiv.org/abs/2406.06369>
- [57] Anas Nadeem, Muhammad Usman Sarwar, and Muhammad Zubair Malik. 2021. Automatic issue classifier: A transfer learning framework for classifying issue reports. In *Proceedings of the 2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 421–426. DOI: <https://doi.org/10.1109/ISSREW53611.2021.00113>
- [58] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Nick Barnes, and Ajmal Mian. 2023. A comprehensive overview of large language models. arXiv:2307.06435. Retrieved from <https://arxiv.org/abs/2307.06435>
- [59] Wendkuuni C. Ouedraogo, Kader Kabore, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawende F. Bissyande. 2024. LLMs and prompting for unit test generation: A large-scale evaluation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. ACM, New York, NY, 2464–2465. DOI: <https://doi.org/10.1145/3691620.3695330>
- [60] Wendkuuni C. Ouedraogo, Yinghua Li, Kader Kabore, Xunzhu Tang, Anil Koyuncu, Jacques Klein, David Lo, and Tegawende F. Bissyandé. 2024. Test smells in LLM-generated unit tests. arXiv:2410.10628. DOI: <https://doi.org/10.48550/arXiv.2410.10628>
- [61] Quentin Perez, Pierre-Antoine Jean, Christelle Urtado, and Sylvain Vauttier. 2021. Bug or not bug? That is the question. In *Proceedings of the 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 47–58.
- [62] Oskar Picus and Camelia Serban. 2022. Bugsby: A tool support for bug triage automation. In *Proceedings of the 2nd ACM International Workshop on AI and Software Testing/Analysis*, 17–20.
- [63] Natthakul Pingclasai, Hideaki Hata, and Ken-ichi Matsumoto. 2013. Classifying bug reports to bugs and other requests using topic modeling. In *Proceedings of the 2013 20th Asia-Pacific Software Engineering Conference (APSEC)*, Vol. 2. IEEE, 13–18.
- [64] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.
- [65] Gokul Rejithkumar, Preethu Rose Anish, and Smita Ghaisas. 2024. Text-to-text generation for issue report classification. In *Proceedings of the 3rd ACM/IEEE International Workshop on NL-based Software Engineering*, 53–56.
- [66] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv:2402.07927. Retrieved from <https://arxiv.org/abs/2402.07927>
- [67] Carolyn B. Seaman, Forrest Shull, Myrna Regardie, Denis Elbert, Raimund L. Feldmann, Yuepu Guo, and Sally Godfrey. 2008. Defect categorization: Making use of a decade of widely varying historical data. In *Proceedings of the 2nd ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, 149–157.
- [68] Murray Shanahan. 2024. Talking about large language models. *Communications of the ACM* 67, 2 (2024), 68–79.
- [69] Victor S. Sheng, Foster Provost, and Panagiotis G. Ipeirotis. 2008. Get another label? Improving data quality and data mining using multiple, noisy labelers. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, Las Vegas Nevada, 614–622. DOI: <https://doi.org/10.1145/1401890.1401965>
- [70] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2023. Prompt engineering or fine tuning: An empirical assessment of large language models in automated software engineering tasks. arXiv:2310.10508. DOI: <https://doi.org/10.48550/arXiv.2310.10508>
- [71] Chenglei Si, Zhe Gan, Zhengyuan Yang, Shuohang Wang, Jianfeng Wang, Jordan Boyd-Graber, and Lijuan Wang. 2022. Prompting gpt-3 to be reliable. arXiv:2210.09150. Retrieved from <https://arxiv.org/abs/2210.09150>
- [72] Mohammed Latif Siddiq and Joanna C. S. Santos. 2022. Bert-based github issue report classification. In *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering*, 33–36.
- [73] Hao Sun, Alihan Hüyük, and Mihaela van der Schaar. 2023. Query-dependent prompt evaluation and optimization with offline inverse RL. In *Proceedings of the 12th International Conference on Learning Representations*.

- [74] Lin Tan, Chen Liu, Zhenmin Li, Xuanhui Wang, Yuanyuan Zhou, and Chengxiang Zhai. 2014. Bug characteristics in open source software. *Empirical Software Engineering* 19 (2014), 1665–1705.
- [75] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. arXiv:2408.00118. Retrieved from <https://arxiv.org/abs/2408.00118>
- [76] Ferdian Thung, David Lo, and Lingxiao Jiang. 2012. Automatic defect categorization. In *Proceedings of the 2012 19th Working Conference on Reverse Engineering*. IEEE, 205–214.
- [77] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from <https://arxiv.org/abs/2302.13971>
- [78] Alexander Trautsch and Steffen Herbold. 2023. Predicting issue types with seBERT. In *Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering (NLBSE '22)*. ACM, New York, NY, 37–39. DOI: <https://doi.org/10.1145/3528588.3528661>
- [79] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [80] Anthony J. Viera and Joanne M. Garrett. 2005. Understanding interobserver agreement: The kappa statistic. *Family Medicine* 37, 5 (2005), 360–363.
- [81] Andreas Vogelsang and Jannik Fischbach. 2024. Using large language models for natural language processing tasks in requirements engineering: A systematic guideline. arXiv:2402.13823. Retrieved from <https://arxiv.org/abs/2402.13823>
- [82] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (Apr. 2024), 911–936. DOI: <https://doi.org/10.1109/TSE.2024.3368208>
- [83] Jun Wang, Xiaofang Zhang, and Lin Chen. 2021. How well do pre-trained contextual language representations recommend labels for github issues? *Knowledge-Based Systems* 232 (Nov. 2021), 107476. DOI: <https://doi.org/10.1016/j.knsys.2021.107476>
- [84] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. arXiv:2305.04091. Retrieved from <https://arxiv.org/abs/2305.04091>
- [85] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. arXiv:2203.11171. Retrieved from <https://arxiv.org/abs/2203.11171>
- [86] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, Vol. 35, 24824–24837.
- [87] Xiaoxue Wu, Wei Zheng, Xiang Chen, Yu Zhao, Tingting Yu, and Dejun Mu. 2021. Improving high-impact bug report prediction with combination of interactive machine learning and active learning. *Information and Software Technology* 133 (2021), 106530.
- [88] Xiaoxue Wu, Wei Zheng, Xin Xia, and David Lo. 2022. Data quality matters: A case study on data label correctness for security bug report prediction. *IEEE Transactions on Software Engineering* 48, 7 (Jul. 2022), 2541–2556. DOI: <https://doi.org/10.1109/TSE.2021.3063727>
- [89] Xin Xia, David Lo, Xinyu Wang, and Bo Zhou. 2014. Automatic defect categorization based on fault triggering conditions. In *Proceedings of the 2014 19th International Conference on Engineering of Complex Computer Systems*. IEEE, 39–48.
- [90] Xiaoyuan Xie, Yuhui Su, Songqiang Chen, Lin Chen, Jifeng Xuan, and Baowen Xu. 2021. MULA: A just-in-time multi-labeling system for issue reports. *IEEE Transactions on Reliability* 71, 1 (2021), 250–263.
- [91] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. arXiv:2407.10671. Retrieved from <https://arxiv.org/abs/2407.10671>
- [92] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. arXiv:2305.10601. DOI: <https://doi.org/10.48550/arXiv.2305.10601>
- [93] Gokul Yenduri, M. Ramalingam, G. Chemmalar Selvi, Y. Supriya, Gautam Srivastava, Praveen Kumar Reddy Maddikunta, G. Deepti Raj, Rutvij H. Jhaveri, B. Prabadevi, Weizheng Wang, et al. 2024. GPT (generative pre-trained transformer)—a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. *IEEE Access* 12 (2024), 54608–54649.
- [94] Daiki Yokoyama, Kinari Nishiura, and Akito Monden. 2024. Identifying security bugs in issue reports: Comparison of BERT, N-gram IDF and ChatGPT. In *Proceedings of the 2024 IEEE/ACIS 22nd International Conference on Software*

- Engineering Research, Management and Applications (SERA)*, 328–333. DOI : <https://doi.org/10.1109/SERA61261.2024.10685583>
- [95] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. arXiv:2205.10625. Retrieved from <https://arxiv.org/abs/2205.10625>
 - [96] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2022. *Large language models are human-level prompt engineers*. arXiv:2211.01910. Retrieved from <https://arxiv.org/abs/2211.01910>
 - [97] Yu Zhou, Yanxiang Tong, Ruihang Gu, and Harald Gall. 2016. Combining text mining and data mining for bug report classification. *Journal of Software: Evolution and Process* 28, 3 (2016), 150–176.
 - [98] Banghua Zhu, Evan Frick, Tianhao Wu, Hanlin Zhu, Karthik Ganesan, Wei-Lin Chiang, Jian Zhang and Jiantao Jiao. 2024. Starling-7B: Improving LLM helpfulness & harmlessness with RLAI. In *First Conference on Language Modeling*. Retrieved from <https://openreview.net/forum?id=GqDntYTTbk>
 - [99] Minhaz F. Zibran. 2016. On the effectiveness of labeled latent dirichlet allocation in automatic bug-report categorization. In *Proceedings of the 38th International Conference on Software Engineering Companion*, 713–715.
 - [100] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. arXiv:1909.08593. Retrieved from <https://arxiv.org/abs/1909.08593>
 - [101] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schroter, and C. Weiss. 2010. What makes a good bug report? *IEEE Transactions on Software Engineering* 36, 5 (Sep. 2010), 618–643. DOI : <https://doi.org/10.1109/TSE.2010.63>
 - [102] Ahmet Üstün, Viraat Aryabumi, Zheng Yong, Wei-Yin Ko, Daniel D’souza, Gbemileke Onilude, Neel Bhandari, Shivalika Singh, Hui-Lee Ooi, Amr Kayid, et al. 2024. Aya model: An instruction finetuned open-access multilingual language model. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.), *Long Papers*, Vol. 1, Association for Computational Linguistics, 15894–15939. DOI : <https://doi.org/10.18653/v1/2024.acl-long.845>

Received 21 November 2024; revised 4 April 2025; accepted 14 May 2025